智能搬运小车
STM32 整车程序讲解

工程：SmartCar_F407 · 正点原子探索者 STM32F407ZGT6

幻尔四路编码电机驱动板 · 麦克纳姆轮 · ST3215 ×3 + SC09 机械臂

2026 年 10 月 13 日

本讲解面向初学者：从"程序整体在干什么"讲起，再逐个文件讲代码，最后讲上车怎么调试。
文中代码片段直接摘自工程源码。

目录

1. 怎么读这份讲解（先补几个 C 语言知识点）
2. 程序整体在做什么：比赛流程与程序分层
3. 从上电到待机：main.c 与 app.c
4. 程序的"心跳"：sys_poll 主循环
5. 硬件抽象层 port：引脚、串口中断、环形缓冲区
6. 电机驱动板通信 motor_drv：I2C 与寄存器
7. 麦轮底盘 chassis：运动学、里程计、速度量化
8. 场地地图 field：坐标系、区域、停车位置怎么算
9. 走位与路线规划 nav
10. 舵机通信 bus_servo：飞特协议逐字节拆解
11. 机械臂 arm：逆运动学与取放动作
12. 视觉通信 vision：协议与"理想点+偏差"
13. 任务码 task_code：156+123+516+231 怎么解析
14. 整场流程 mission：逐段对照比赛规则
15. 屏幕 display、陀螺仪 imu、调试命令 console
16. 电脑模拟器：它模拟了什么、结果怎么看
17. 上车调试完整步骤
18. 常见问题排查
19. 参数速查表（robot_config.h）
20. 已知风险与下一步

1　怎么读这份讲解

建议阅读顺序：第 2 章（整体）→ 第 14 章（整场流程，最像"说明书"）→ 第 4 章（主循环）→ 其余各章按需要看。 第 17 章是上车调试的操作步骤，真正上车时照着做。

整个工程约 2700 行我们自己写的代码（不含 ST 的 HAL 库），分成 15 个 .c 文件，每个文件只负责一件事。 不需要一次全看懂——先知道"每个文件管什么"，用到哪块再细看哪块。

1.1 读这套代码需要的几个 C 语言知识点

写法	意思	本工程里的例子
`#define 名字 值`	宏定义：编译前把"名字"原样替换成"值"。用来集中放参数，改一处全局生效	`#define WHEEL_DIAMETER_MM 60.0f`
`typedef struct {...} 名字;`	结构体：把几个相关的变量打包成一个新类型	`pose_t` 里有 x、y、th 三个数，表示"位置+朝向"
`static`（函数外）	这个变量/函数只在本文件内可见，别的文件碰不到，避免互相干扰	`static pose_t s_pose;` 只有 chassis.c 能直接改车的位置
`static`（函数内）	变量只初始化一次，函数退出后值保留	world.c 里记录"上一次停在哪个物料"
`const`	只读，不能改	场地坐标表 `static const point_t ROUGH_RING[3]`
`指针 *p`、`&x`	`&x` 取 x 的地址；函数参数是指针时，函数可以把结果"写回"给调用者	`arm_grip_close(&has)` 让函数把"夹没夹到"写进 has
`1.0f`	后面的 f 表示单精度 float。F407 有硬件单精度浮点运算单元，用 float 很快，用 double 会慢很多	全工程都用 float
`uint8_t / int32_t`	明确位数的整数：u=无符号，8/16/32 是位数	串口字节是 `uint8_t`，编码器累计值是 `int32_t`
`a ? b : c`	如果 a 成立取 b，否则取 c	`(zone == 2) ? 150.0f : 2250.0f`
`(uint32_t)(now - t0) > 500`	判断"过去了 500ms 没有"。用减法再转无符号，即使计时器溢出回零也不会算错	所有超时判断都这样写

1.2 单位和命名约定

- 长度一律 **毫米 mm**，时间 **毫秒 ms**，角度在计算里用 **弧度**（参数文件里写度数的会带 `_DEG`，用 `DEG2RAD()` 换算）。
- 以 `s_` 开头的变量是"本文件私有的状态"，例如 `s_pose`。
- 函数名以文件名开头：`chassis_xxx` 在 chassis.c，`arm_xxx` 在 arm.c，找起来很方便。
- 参数注释里带【标定】的，都必须上车实测后修改；带【现场】的，到比赛场地实测后修改。

2　程序整体在做什么

2.1 比赛要求我们做什么（初赛）

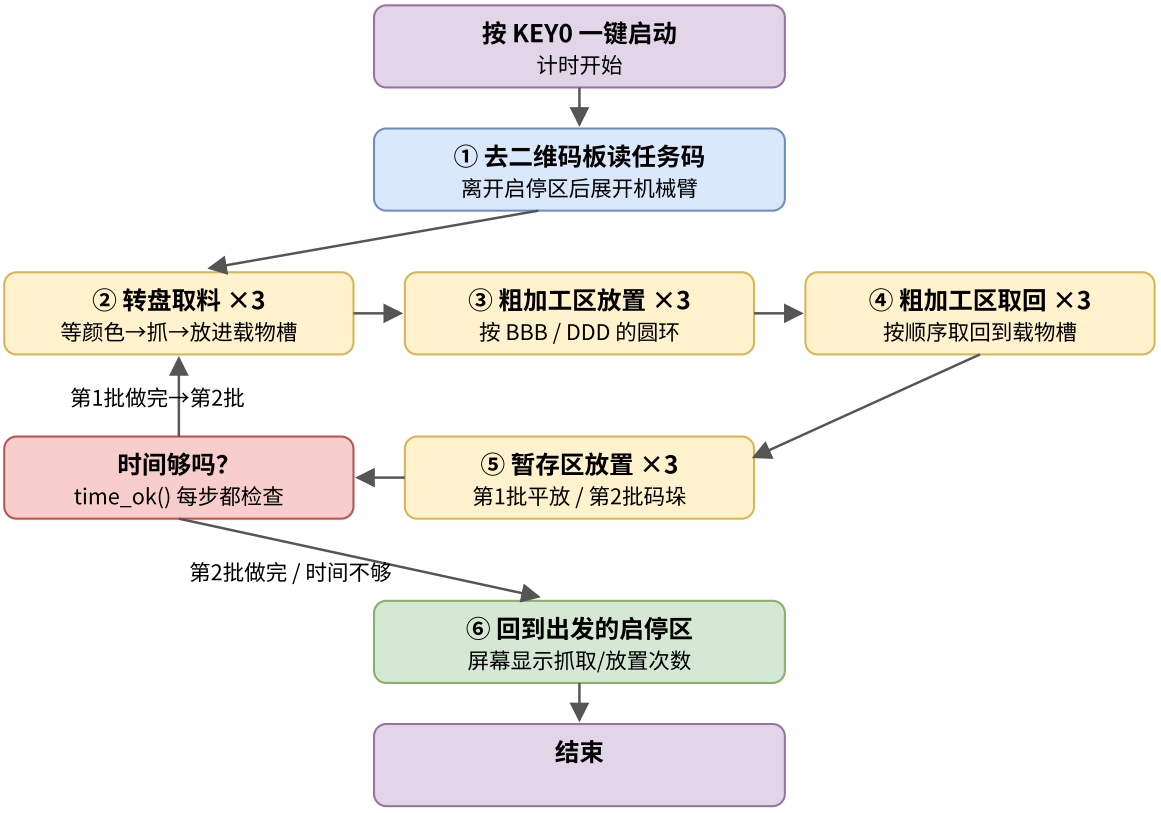


图 2-1　整场比赛流程（对应 mission.c）

按规则第 4 节"现场初赛"：一键启动后，小车先到二维码板读任务码并显示；然后到原料转盘**按顺序**逐个抓第一批 3 个物料，**每抓一个必须先放到车上**才能抓下一个；带着物料到粗加工区按指定圆环放下，再按顺序取回车上；到暂存区放到指定圆环；第二批同样流程，最后在暂存区**叠放到第一批同颜色物料上**（码垛）；回到出发的启停区，屏幕显示抓取/放置正确次数。全程限时 **3 分钟**。

> **两条特别容易犯规的规则**，程序里专门处理了：
> ① 跨区域行驶时，物料必须在车上的载物槽里，不能夹在手爪里运（"本区域内不受限制"）。
> ② 车身离开启停区后只能在灰色车道行驶，进入其他颜色区域（禁行区、加工区等）比赛结束——机械臂伸出去不算。

2.2 程序分成几层

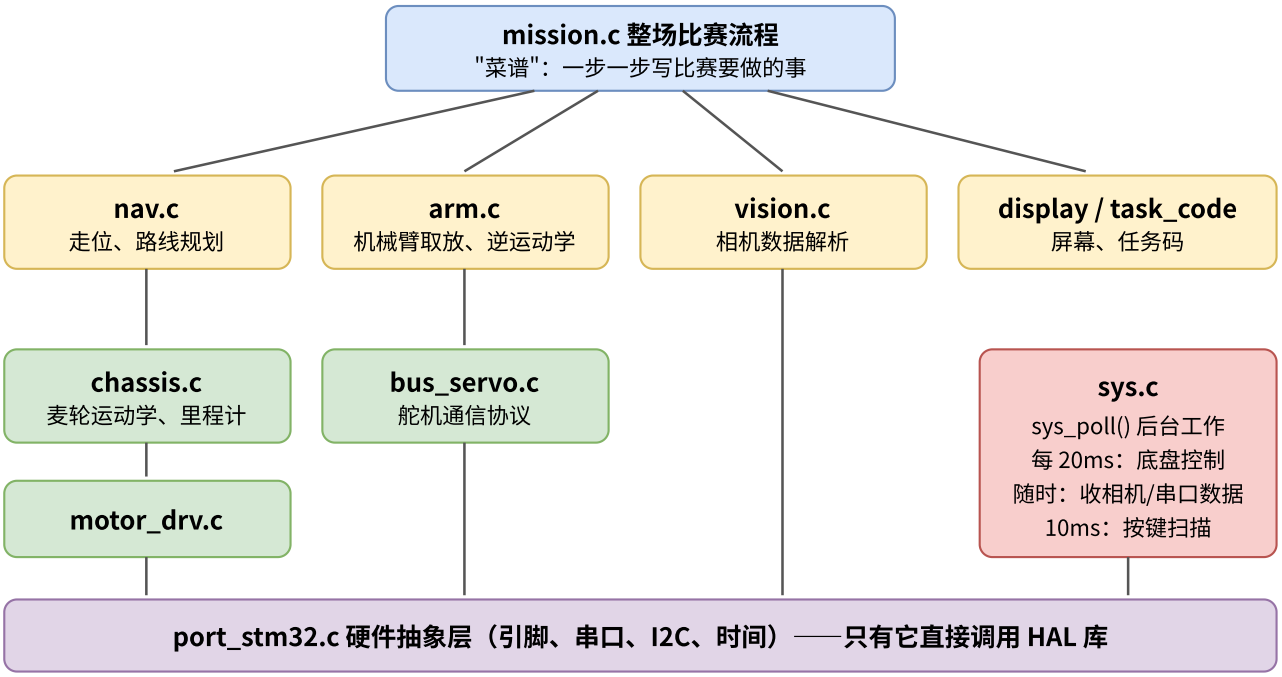


图 2-2　程序分层：上层只调用下层，不跨层乱调

可以把程序想成一家餐厅：**mission.c 是菜谱**（先做什么后做什么）；**nav、arm、vision 是几位厨师**（会"开到某处""夹起某物""看一眼"）；**chassis、bus_servo、motor_drv 是厨具的使用方法**（轮子怎么转、舵机怎么通信）；最底下的 **port_stm32.c 是厨房的水电**（具体哪根引脚、哪个串口）。

这样分层的好处：

- **换硬件只改底层**。例如舵机串口从 USART2 换成 USART4，只改 port_stm32.c；比赛流程完全不用动。
- **能在电脑上模拟**。把 port_stm32.c 换成 sim/port_sim.c，上面所有代码原封不动在电脑上跑，用来验证比赛逻辑（第 16 章）。
- **出问题好定位**。车不走 → 先用调试命令单独测 motor_drv；轮子能转但走不准 → 看 chassis 标定；走位对但放不准 → 看 arm。

2.3 两种"写法"的取舍：顺序写法 + 后台轮询

单片机程序常见两种写法：一种是"状态机"（把流程拆成很多状态，每次进来判断当前在哪个状态），另一种是"顺序写法"（像菜谱一样从上往下写）。状态机效率高但很难读；本工程选了**顺序写法**，比赛流程读起来就像规则原文：

mission.c · mission_run() 的核心

```
for (int b = 0; b < 2 && !sys_aborted(); b++) {   /* 两批 */
    turntable_batch(b);     /* 转盘取料 */
    rough_place(b);         /* 粗加工区放置 */
    rough_retrieve(b);      /* 粗加工区取回 */
    stash_place(b);         /* 暂存区放置 / 码垛 */
    if (R->aborted_by_time) break;
}
```

顺序写法的难点是：当程序"停在某一步等待"时（比如等机械臂转到位），底盘控制、接收相机数据这些事谁来做？ 答案是第 4 章的 `sys_poll()`：**所有等待的地方都在不停地调用它**，它负责把这些"后台工作"按时做掉。

3　从上电到待机：main.c 与 app.c

main.c · main()

```
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    app_main();
    while (1) {}
}
```

上电后芯片先执行启动文件 `startup_stm32f407zgtx.s`（设置栈、把初值拷到内存），然后进入 `main()`。main 只做三件事：

1. `HAL_Init()`：初始化 HAL 库，并开启每 1ms 一次的系统节拍（SysTick 中断里 `HAL_IncTick()` 让毫秒计数 +1）。
2. `SystemClock_Config()`：把主频从默认 16MHz 提到 168MHz。
3. `app_main()`：进入我们自己的程序，永不返回。

3.1 为什么要 168MHz？和点灯程序有什么不同

你之前的点灯工程用的是芯片内部 16MHz 时钟（HSI），没开锁相环（PLL）。点灯够用，但这里不行：舵机串口要跑 1Mbps，浮点运算多，16MHz 太慢也不够精确。探索者板上有一颗 8MHz 外部晶振（HSE），经 PLL 倍频：

main.c · PLL 参数

```
    osc.PLL.PLLM = 8;
    osc.PLL.PLLN = 336;
    osc.PLL.PLLP = RCC_PLLP_DIV2;
    osc.PLL.PLLQ = 7;
```

8MHz ÷ 8 × 336 ÷ 2 = **168MHz**。外设总线 APB1 = 168/4 = 42MHz（USART2、USART3、I2C1 挂在这），APB2 = 168/2 = 84MHz（USART1、USART6）。串口波特率由总线时钟分频得到：42MHz÷16÷1M = 2.625，84MHz÷16÷1M = 5.25，都能精确分出 1Mbps，所以舵机两条串口选了 USART2 和 USART6。

> **BOOT0 跳线：**你在点灯时遇到"复位后停在内部启动程序、要靠调试脚本才能进 main"，就是因为 BOOT0 不是低电平。把 BOOT0 跳线接到 GND，芯片上电才会从 Flash 运行我们的程序。
>
> `stm32f4xx_hal_conf.h` 里的 `HSE_VALUE` 已改为 8000000。

3.2 app_main：初始化 → 待机

app.c · app_main()

```
void app_main(void)
{
    port_init();
    sys_init();
    LOG("==== 智能搬运小车 启动 ====");
```

```
    field_init();
    chassis_init();
    vision_init();
    display_init();

    if (motor_drv_init() != 0) LOG("警告：电机驱动板未就绪，底盘不会动");
    if (imu_init() == 0) LOG("陀螺仪已启用：朝向用陀螺仪测量");
    if (arm_init() != 0)       LOG("警告：有舵机未应答，机械臂可能不完整");

    int zone = 1;
    show_ready(zone);
    LOG("待机：KEY1 切换启停区，KEY0 启动；串口输入 help 查看调试命令");
    beep(80);

    uint32_t last_blink = 0;
    bool led = false;
    for (;;) {
        sys_poll();

        if (sys_key_pressed(PORT_KEY_ZONE)) {
            zone = (zone == 1) ? 2 : 1;
            show_ready(zone);
            LOG("出发启停区 = %d", zone);
            beep(40);
        }

        if (sys_key_pressed(PORT_KEY_START)) {
            beep(150);
            mission_cfg_t cfg = { zone, false, "" };
            mission_result_t res;
            imu_track_bias(false);          /* 出发后车在动，不能再修正零漂 */
            mission_run(&cfg, &res);
            imu_track_bias(true);
            beep(400);
        }

        console_execute_pending();

        /* 绿灯慢闪表示程序在正常运行 */
        uint32_t now = sys_now();
        if ((uint32_t)(now - last_blink) > 500) {
            last_blink = now;
            led = !led;
            port_led(1, led);
        }
    }
}
```

读法：

- 前半段依次初始化各模块。驱动板、舵机没应答只打印警告，不卡死——这样即使只接了一部分硬件，也能用串口命令调试已接好的部分。
- `for (;;)` 是死循环（待机）。每圈先 `sys_poll()` 做后台工作，再看按键：KEY1 切换启停区 1/2，KEY0 一键启动 `mission_run()`。
- `console_execute_pending()`：执行电脑串口发来的调试命令（第 15 章）。
- 绿灯每 500ms 翻转一次：看到绿灯慢闪就说明程序在正常跑。

4　程序的"心跳"：sys_poll 主循环

sys.c · sys_poll()

```
void sys_poll(void)
{
    if (s_in_poll) { port_idle(); return; }   /* 防止重入：后台工作里如果又调用了等待函数，不会无限嵌套 */
    s_in_poll = true;

    uint32_t now = port_millis();

    /* 1) 底盘控制：固定 20ms 周期 */
    if ((uint32_t)(now - s_last_ctrl) >= CTRL_PERIOD_MS) {
        uint32_t dt = now - s_last_ctrl;
        if (dt > 100) dt = CTRL_PERIOD_MS;        /* 长时间阻塞后不要用异常大的 dt */
        s_last_ctrl = now;
        nav_update();                             /* 走位控制器：算出这个周期的目标速度 */
        chassis_update((float)dt / 1000.0f);      /* 底盘：读编码器、下发轮速 */
    }

    /* 2) 按键：10ms 扫描一次 */
    if ((uint32_t)(now - s_last_key) >= 10) {
        s_last_key = now;
        scan_keys();
    }

    /* 3) 摄像头数据、串口命令、屏幕刷新 */
    vision_poll();
    console_poll();
    display_poll();

    s_in_poll = false;
    port_idle();
}
```

sys_poll 每被调用一次，就检查"有没有到时间该做的事"：

工作	频率	做什么
`nav_update()`	每 20ms	走位控制器：根据"离目标多远"算出车该用多快的速度
`chassis_update()`	每 20ms	读四个编码器 → 更新车的位置；把速度换算成四个轮速发给驱动板
`scan_keys()`	每 10ms	按键消抖：连续 3 次（30ms）读到同样状态才算数，防止抖动误触发
`vision_poll()`	每次	把相机串口收到的字节拼成一行，解析出二维码/颜色/偏差
`console_poll()`	每次	收电脑串口命令；如果是 `s`（急停）立刻生效
`display_poll()`	有变化时，最快 5 次/秒	刷新屏幕内容

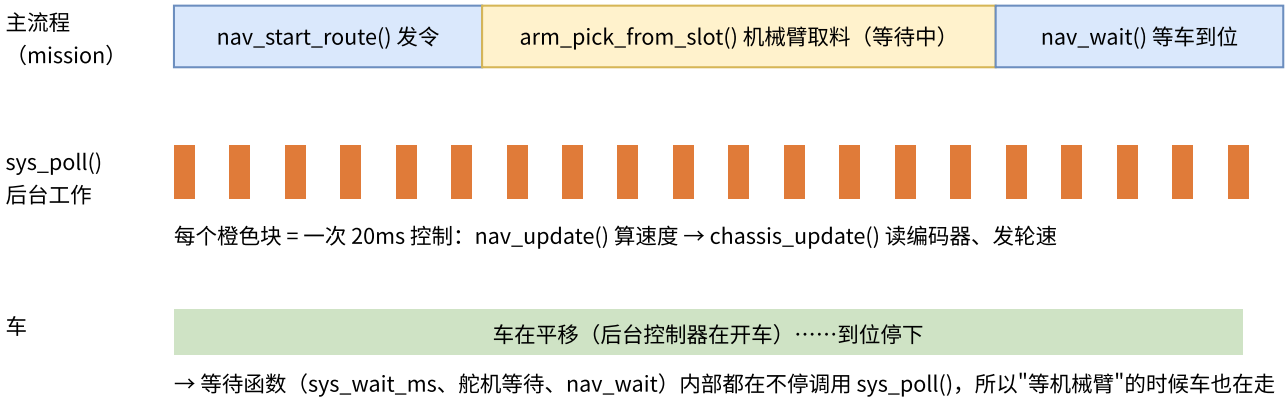


图 4-1　"边开车边动机械臂"时，主流程和后台工作的时间关系

4.1 为什么所有等待都要调用 sys_poll

sys.c · sys_wait_ms()

```
void sys_wait_ms(uint32_t ms)
{
    uint32_t t0 = port_millis();
    while ((uint32_t)(port_millis() - t0) < ms && !s_abort) sys_poll();
}
```

对比点灯程序里的 `HAL_Delay(500)`：它在 500ms 里什么都不干，"死等"。如果小车运动时也这样死等，这 500ms 里底盘控制停了，车会按最后一次的速度一直冲出去！所以本工程**禁止用 HAL_Delay**，一律用 `sys_wait_ms()`：它在等待的同时不停调用 sys_poll，后台工作照常进行。

> **记住一条规则：**以后自己加代码，凡是要"等"的地方（while 循环等某个条件），循环体里一定要调用 `sys_poll()`。

4.2 防止"重入"

sys_poll 开头有个 `s_in_poll` 标志。如果后台工作里某个函数又调用了等待函数（等待函数又调用 sys_poll），标志会让第二层直接返回，避免无限嵌套。`port_idle()` 在单片机上什么都不做；在模拟器里它让仿真时间前进 1ms。

5　硬件抽象层 port：引脚、串口中断、环形缓冲区

`port.h` 声明了上层能用的全部硬件功能（时间、I2C、串口、按键、灯、蜂鸣器），`port_stm32.c` 用 HAL 库实现它们。引脚是对照探索者 V3.4 原理图选的：

功能	引脚	说明
I2C1（电机驱动板、陀螺仪）	PB8 SCL / PB9 SDA	板载 24C02、ST480MC 磁力计也在这条总线上，地址都不冲突
USART1（日志、调试命令）	PA9 / PA10	板载 CH340 USB 转串口，115200
USART2（ST3215 总线）	PA2 TX / PA3 RX	1Mbps；原理图上经 P4 跳线连到 RS232/RS485，**要拔掉 P4 跳线帽**
USART6（SC09 总线）	PC6 TX / PC7 RX	1Mbps；这两个脚也是 OV 摄像头接口的数据线，不插 OV 模块就是空闲的
USART3（视觉模块）	PB10 TX / PB11 RX	115200；经 P2 跳线连到 RS232/蓝牙，**要拔掉 P2 跳线帽**
按键	KEY0 PE4、KEY1 PE3、KEY2 PE2、WK_UP PA0	KEY 按下为低电平，WK_UP 按下为高电平
LED / 蜂鸣器	PF9 红、PF10 绿（低电平亮）、PF8 蜂鸣器	和点灯实验相同

5.1 串口接收：中断 + 环形缓冲区

串口数据随时可能到来（相机每 100ms 发一次，舵机回复只有几十微秒），主程序不可能一直盯着。办法是：**每收到一个字节，硬件触发一次中断**，中断函数把字节存进一个缓冲区；主程序有空时再从缓冲区里取。

port_stm32.c · 环形缓冲区与中断

```
static void rb_put(ringbuf_t *r, uint8_t c)
{
    uint16_t n = (uint16_t)((r->head + 1) % RB_SIZE);
    if (n != r->tail) { r->buf[r->head] = c; r->head = n; }   /* 满了就丢弃 */
}

/* 中断里直接读寄存器，比 HAL 的接收中断更简单可靠 */
static void uart_irq(port_uart_t ch)
{
    USART_TypeDef *u = UART_OF[ch]->Instance;
    uint32_t sr = u->SR;
    if (sr & (USART_SR_RXNE | USART_SR_ORE | USART_SR_FE | USART_SR_NE)) {
        uint8_t c = (uint8_t)u->DR;            /* 读 DR 同时清除 RXNE 和错误标志 */
        if (sr & USART_SR_RXNE) rb_put(&s_rb[ch], c);
    }
}
void USART1_IRQHandler(void) { uart_irq(PORT_UART_LOG); }
void USART2_IRQHandler(void) { uart_irq(PORT_UART_SERVO_ST); }
void USART6_IRQHandler(void) { uart_irq(PORT_UART_SERVO_SC); }
void USART3_IRQHandler(void) { uart_irq(PORT_UART_VISION); }
```

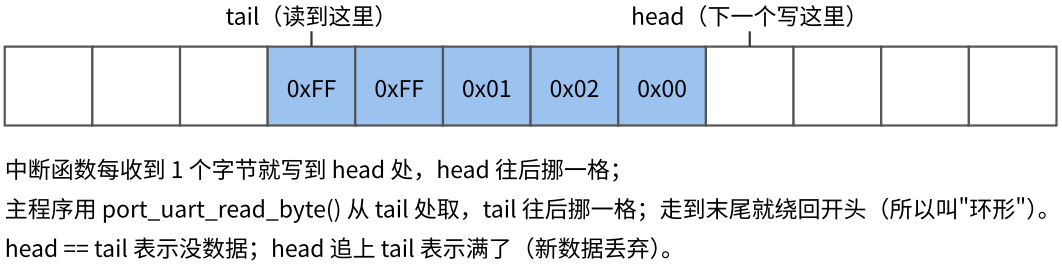


图 5-1　环形缓冲区：写指针 head 和读指针 tail 绕圈追逐

这里没用 HAL 的 `HAL_UART_Receive_IT`，而是直接读寄存器：读 `SR`（状态寄存器）判断有没有数据，读 `DR`（数据寄存器）取出字节。"先读 SR 再读 DR"这个顺序会顺便清掉溢出错误标志（ORE），避免串口因溢出卡死——这是用 HAL 接收中断时初学者常遇到的坑。

5.2 I2C 卡死恢复

I2C 有个经典问题：通信中途单片机复位，从机可能正在发数据、一直把 SDA 拉低，总线就"卡死"了。`i2c_bus_recover()` 在初始化前把 SCL 当普通引脚手动拉高拉低 9 次，让从机把剩下的位发完、释放总线。每次 I2C 读写失败，`i2c_reset()` 会重新初始化 I2C，所以偶尔一次干扰不会让车彻底失控。

6　电机驱动板通信 motor_drv：I2C 与寄存器

幻尔四路驱动板自己带了单片机，负责 PWM 和速度闭环（PID）。我们只需通过 I2C 告诉它"每个电机要多快"，并读回编码器累计值。 I2C 只用两根线（SCL 时钟、SDA 数据），一条总线可以挂很多设备，用 **7 位地址**区分，驱动板地址是 **0x34**。 读写都按"寄存器"进行——可以把驱动板想成一排带编号的格子：

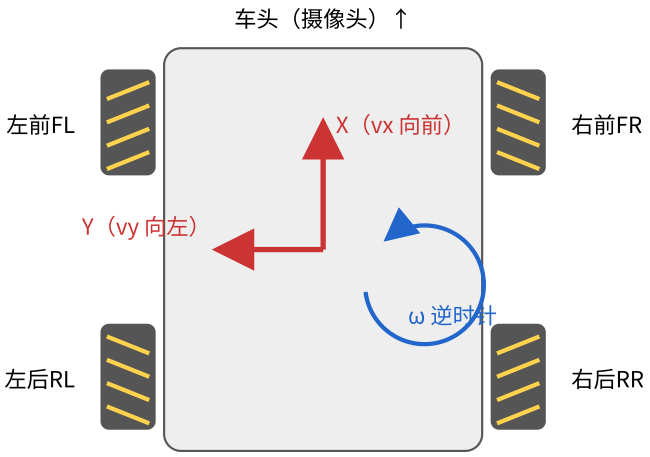
寄存器（十进制）	内容	读/写	格式
0	电池电压	读 2 字节	小端，单位 mV（需实物验证）
20	电机类型	写 1 字节	3 = JGB 霍尔编码器类（25GA370 先按此试）
21	编码器极性	写 1 字节	0/1；电机越转越快停不下来时改另一个值
31	固定 PWM（开环）	写 4 字节	每个电机一个 int8，−100~100
51	固定速度（闭环）	写 4 字节	每个电机一个 int8，单位"编码器脉冲/10ms"
60	编码器累计值	读 16 字节	4 个 int32，小端

motor_drv.c · 读 4 个编码器

```
int motor_drv_read_counts(int32_t cnt[4])
{
    uint8_t b[16];
    if (port_i2c_read(MOTOR_DRV_I2C_ADDR, MDRV_REG_ENC_TOTAL, b, 16)) { s_err++; return -1; }
    for (int i = 0; i < 4; i++) {
        uint32_t u = (uint32_t)b[4 * i] | ((uint32_t)b[4 * i + 1] << 8) |
                     ((uint32_t)b[4 * i + 2] << 16) | ((uint32_t)b[4 * i + 3] << 24);
        cnt[i] = (int32_t)u;
    }
    return 0;
}
```

"小端"是什么？ 一个 32 位整数要拆成 4 个字节传输，小端（little-endian）是低位字节在前。例如编码器值 1000 = 0x000003E8， 传过来的 4 个字节依次是 `E8 03 00 00`。代码里 `b[0] | (b[1] << 8) | (b[2] << 16) | (b[3] << 24)` 就是把它们按位置拼回去：0xE8 + 0x03×256 = 232 + 768 = 1000。最后强制转成 `int32_t`，倒转时的负数也能正确表示。

7　麦轮底盘 chassis：运动学、里程计、速度量化



黄色斜线 = 从上往下看到的顶部滚子（45°）
四个轮顶部滚子连起来呈"O"形（菱形），着地滚子呈 X 形

轮子转动时，地面只能沿"垂直于滚子"的方向推它，
四个轮子的推力斜着交叉，组合起来就能：

- 四轮同向转 → 前进/后退
- 对角两组反向 → 横着走（平移）
- 左右两侧反向 → 原地转
- 三者叠加 → 边走边转、斜着走

a = 左右轮中心距的一半（133mm）
b = 前后轮中心距的一半（78mm）

图 7-1　麦轮原理与车体坐标系（俯视，"O"形安装）

7.1 逆运动学：想让车这样走，四个轮子各转多快

设车想要的速度为：向前 vx、向左 vy（mm/s），逆时针转 ω（rad/s），k = a + b。四个轮子的线速度：

左前 FL = vx − vy − k·ω	右前 FR = vx + vy + k·ω
左后 RL = vx + vy − k·ω	右后 RR = vx − vy + k·ω

验证一下直觉：只往前（vy=ω=0）→ 四个轮子都等于 vx，全部向前转 ✔；只向左平移（vx=ω=0）→ FL、RR 向后，FR、RL 向前，对角两组反向 ✔； 只逆时针转 → 左边两轮向后、右边两轮向前 ✔。

chassis.c · chassis_update() 后半段

```
    /* 逆运动学：车体速度 → 四个轮子线速度 */
    float k = K_ROT / ROT_SCALE;
    float vx = s_out_vx / FWD_SCALE, vy = s_out_vy / STRAFE_SCALE;   /* 打滑多的方向，轮子要多转一点 */
    float w[4];
    w[WHEEL_FL] = vx - vy - k * s_out_w;
    w[WHEEL_FR] = vx + vy + k * s_out_w;
    w[WHEEL_RL] = vx + vy - k * s_out_w;
    w[WHEEL_RR] = vx - vy + k * s_out_w;

    /* 有轮子超过最大速度时，四个轮子按比例一起缩小，保证方向不变 */
    float vmax_wheel = (float)MOTOR_SPEED_LIMIT * MM_PER_COUNT * 100.0f;
    float m = 0;
    for (int i = 0; i < 4; i++) if (fabsf(w[i]) > m) m = fabsf(w[i]);
    if (m > vmax_wheel) for (int i = 0; i < 4; i++) w[i] *= vmax_wheel / m;

    send_wheels(w);
```

代码里还做了两件事：除以 `FWD_SCALE`、`STRAFE_SCALE`（标定得到的打滑补偿：麦轮横移时打滑多，轮子要多转一点才能走够距离）； 以及"等比例缩小"——如果某个轮子算出来超过最大速度，四个轮子按同一比

例一起缩小，**保证行驶方向不变**，只是慢一点。

7.2 正运动学与里程计：轮子转了多少 → 车走了多少

把上面四个式子反过来解，就能由四个轮子实际转过的距离，算出车体的位移（这就是"里程计"）：

$dx = (FL + FR + RL + RR) / 4$
$dy = (-FL + FR + RL - RR) / 4$
$d\theta = (-FL + FR - RL + RR) / (4k)$

chassis.c · update_odometry()

```
static void update_odometry(float dt)
{
    int32_t cnt[4];
    if (!motor_drv_ok() || motor_drv_read_counts(cnt) != 0) return;
    if (!s_have_cnt) { memcpy(s_last_cnt, cnt, sizeof cnt); s_have_cnt = true; return; }

    float d[4];      /* 每个轮子本周期前进了多少 mm */
    for (int i = 0; i < 4; i++) {
        int ch = WHEEL_CH[i];
        int32_t dc = (cnt[ch] - s_last_cnt[ch]) * WHEEL_SIGN[i];
        s_wheel_total[i] += dc;
        d[i] = (float)dc * MM_PER_COUNT;
    }
    memcpy(s_last_cnt, cnt, sizeof cnt);

    /* 正运动学：由四轮位移求车体位移（车体坐标） */
    float dx  = ( d[WHEEL_FL] + d[WHEEL_FR] + d[WHEEL_RL] + d[WHEEL_RR]) * 0.25f * FWD_SCALE;
    float dy  = (-d[WHEEL_FL] + d[WHEEL_FR] + d[WHEEL_RL] - d[WHEEL_RR]) * 0.25f * STRAFE_SCALE;
    float dth = (-d[WHEEL_FL] + d[WHEEL_FR] - d[WHEEL_RL] + d[WHEEL_RR]) * 0.25f / K_ROT * ROT_SCAL
E;

    /* 有陀螺仪就用陀螺仪的角度（更准），否则用轮子算出来的 */
    float yaw;
    float th_new = s_pose.th + dth;
    if (s_imu_ref && imu_get_yaw(&yaw)) th_new = wrap_pi(yaw + s_imu_offset);
    float th_mid = s_pose.th + wrap_pi(th_new - s_pose.th) * 0.5f;

    /* 转到场地坐标累加（用中间时刻的朝向，转弯时更准） */
    s_pose.x += cosf(th_mid) * dx - sinf(th_mid) * dy;
    s_pose.y += sinf(th_mid) * dx + cosf(th_mid) * dy;
    s_pose.th = wrap_pi(th_new);

    if (dt > 0.0f) { s_meas_vx = dx / dt; s_meas_vy = dy / dt; s_meas_w = dth / dt; }
}
```

要点：

- **每个轮子转 1 个脉冲走多远：** $\pi \times 60 \div 1496 \approx$ **0.126mm**（1496 是暂定的每圈脉冲数，要标定）。
- dx、dy 是**车体坐标**（以车头为前），要旋转到**场地坐标**再累加。用"这一小段的中间朝向"`th_mid` 去旋转，转弯时更准。
- 有陀螺仪时，朝向直接用陀螺仪的角度（第 15 章），因为轮子打滑对朝向的影响最大。
- 车的位置存在 `s_pose` 里，`chassis_pose()` 读取，`chassis_set_pose()` 在出发时和相机校正时改写。

7.3 速度"量化"与小数累积

chassis.c · send_wheels()

```
/* 把四个轮子的线速度(mm/s)换成驱动板单位(脉冲/10ms)发出去 */
static void send_wheels(const float wheel_mmps[4])
{
    int8_t ch_speed[4] = { 0, 0, 0, 0 };
    for (int i = 0; i < 4; i++) {
        float u = wheel_mmps[i] / MM_PER_COUNT * 0.01f;   /* 脉冲/10ms，可能带小数 */
        /*
         * 驱动板只收整数，1 个单位约等于 12.6mm/s，低速时误差很大。
         * 办法：把每次舍掉的小数部分累积起来，攒够 1 就多发 1——平均下来速度就准了。
         */
        u += s_frac[i];
        float r = roundf(u);
        s_frac[i] = clampf(u - r, -1.0f, 1.0f);
        int v = (int)r;
        if (v >  MOTOR_SPEED_LIMIT) v =  MOTOR_SPEED_LIMIT;
        if (v < -MOTOR_SPEED_LIMIT) v = -MOTOR_SPEED_LIMIT;
        ch_speed[WHEEL_CH[i]] = (int8_t)(v * WHEEL_SIGN[i]);
    }
    motor_drv_set_speed(ch_speed);
}
```

驱动板的速度单位是"脉冲/10ms"，而且只收整数。1 个单位 = 100 脉冲/秒 × 0.126mm ≈ **12.6mm/s**，这个分辨率很粗：想要 50mm/s，相当于 3.97 个单位，四舍五入发 4，速度误差还不大；想要 20mm/s = 1.59 单位，发 2 就快了 26%。

解决办法是"把舍掉的小数攒起来"：这次想发 3.97 → 发 4，记下欠了 −0.03；下次想发 3.97 + (−0.03) = 3.94 → 还发 4，欠 −0.06…… 欠到 −0.5 以下时就发一次 3。**平均下来，速度和想要的完全一样**。这种技巧叫 sigma-delta / 误差扩散，舵机、LED 调光里也常用。

7.4 加速度限制

`chassis_update()` 前半段每 20ms 最多把速度改变 `NAV_ACC_MMPS2 × 0.02` = 700 × 0.02 = 14mm/s。效果是起步、刹车都是平滑的斜坡：轮子不容易打滑，载物槽里的物料也不会被甩倒。

8　场地地图 field：坐标系、区域、停车位置

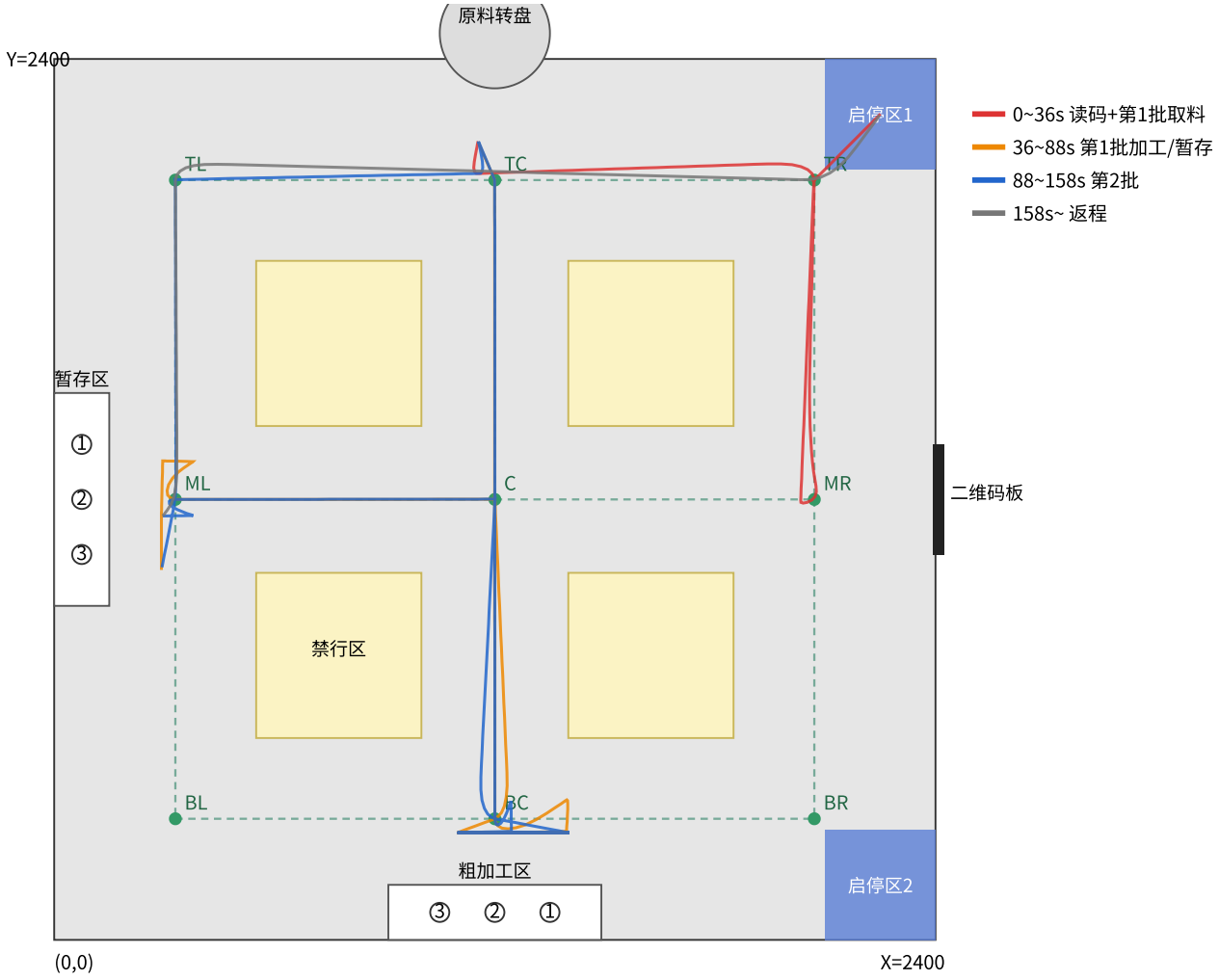


图 8-1　场地坐标（按规则图 1）、车道网格（绿色虚线），以及模拟器里一整场的真实行驶轨迹（任务码 156+123+516+231，启停区 1）

场地坐标原点在左下角，X 向右、Y 向上，单位 mm。`field.c` 里记录了所有区域的位置：四块禁行区、粗加工区、暂存区、转盘、二维码板、 六个圆环中心。这些数字是按规则图估的，**到比赛现场一定要实测再改**，尤其是圆环的编号方向（1 号环在左边还是右边）。

8.1 车道网格

车只沿 9 个"节点"之间的 12 条连线走（图中绿色虚线）：外圈一圈 + 中间十字。外圈车道中心离场地边 330mm， 这个数是这样定的：车宽一半 148mm + 加工区宽 150mm + 余量 32mm。所以沿外圈经过加工区旁边时，车身不会压进加工区。

8.2 已知要够到的目标点，车停在哪？——pose_for_target

field.c · pose_for_target()

```
/*
 * 已知"要让机械臂够到的目标点 tgt"和"车头朝向 th"，
 * 求车停在哪里，能让目标点落在车体坐标 (Xt, Yt) 处。
 * 取 Yt = 回转轴的 Y，使目标正好在回转轴的正前方 → 机械臂基本不用转，误差最小。
 */
static pose_t pose_for_target(point_t tgt, float th, float Xt, float Yt)
{
    pose_t p;
    float c = cosf(th), s = sinf(th);
    p.x = tgt.x - (c * Xt - s * Yt);
    p.y = tgt.y - (s * Xt + c * Yt);
    p.th = th;
    return p;
}
```

这是一个"倒推"：我们希望目标点（例如 2 号环中心）出现在车体坐标的 (Xt, Yt) 处，其中 Yt 取机械臂回转轴的 Y（−45mm），这样目标正好在机械臂的正前方，回转关节几乎不用转，误差最小。已知车头朝向 th，就能反算出车中心该停在哪。

以粗加工区为例（`field_place_pose`）：车头朝 −Y（朝下），车头离区域边线留 `ZONE_GAP_MM` = 25mm，车中心 Y = 150 + 25 + 117 = 292mm。圆环中心 Y = 75，所以 Xt = 217mm——机械臂要往前伸 217 − 45 = 172mm，在够得着的范围内。

> ZONE_GAP 为什么是 25mm 而不是更小？模拟时发现：陀螺仪漂移让车头偏了 2°，车角就会多扫出 148×sin2° ≈ 5mm，再加上定位误差，12mm 的间隙会压线。25mm 是留够余量后的值。

8.3 判断车身有没有压线：field_pose_on_lane

在车身矩形的外框上均匀取 24 个点，逐个换算到场地坐标，看有没有落在场外、禁行区、加工区或转盘里。带 `_m` 的版本会先把车身放大 margin 毫米再检查——规划路线时用 30mm 余量，因为实际行驶会偏离计划路线几毫米到几十毫米。

9　走位与路线规划 nav

9.1 走到一个点：每 20ms 算一次速度

nav.c · nav_update()

```
/* 每个控制周期算一次速度（原理见文件开头） */
void nav_update(void)
{
    if (!s_active) return;
    if (sys_aborted()) { s_result = NAV_ABORTED; nav_cancel(); return; }
    seg_t *g = &s_seg[s_iseg];
    if ((uint32_t)(sys_now() - s_seg_t0) > NAV_TIMEOUT_MS) {
        pose_t p = chassis_pose();
        LOG("nav: 超时 目标(%d,%d) 当前(%d,%d)", (int)g->p.x, (int)g->p.y, (int)p.x, (int)p.y);
        s_result = NAV_TIMEOUT;
        nav_cancel();
        return;
    }
    pose_t p = chassis_pose();
    float ex = g->p.x - p.x, ey = g->p.y - p.y;
    float dist = hypotf(ex, ey);
    float eth = wrap_pi(g->p.th - p.th);

    bool done = false;
    if (g->pass) {
        done = dist < g->pass_tol;                          /* 途经点：到附近就转下一段，不停车 */
    } else if (dist < NAV_POS_TOL_MM && fabsf(eth) < DEG2RAD(NAV_ANG_TOL_DEG)) {
        chassis_set_velocity(0, 0, 0);
        done = ++s_settled >= 3;                            /* 连续 3 个周期都在容差内才算停稳 */
        if (!done) return;
    } else {
        s_settled = 0;
    }
    if (done) {
        if (++s_iseg >= s_nseg) { s_active = false; s_nseg = 0; if (!g->pass) chassis_set_velocity
(0, 0, 0); return; }
        s_settled = 0;
        s_seg_t0 = sys_now();
        g = &s_seg[s_iseg];
        ex = g->p.x - p.x; ey = g->p.y - p.y; dist = hypotf(ex, ey); eth = wrap_pi(g->p.th - p.th);
    }

    /* 线速度大小：越近越慢，并保证来得及刹车 */
    float v = g->vmax;
    float v_brake = sqrtf(g->v_exit * g->v_exit + 2.0f * NAV_ACC_MMPS2 * 0.8f * dist);
    if (v > v_brake) v = v_brake;
    if (!g->pass && v > NAV_KP_POS * dist) v = NAV_KP_POS * dist;
    if (dist > NAV_POS_TOL_MM * 0.5f && v < MIN_SPEED_MMPS) v = MIN_SPEED_MMPS;
    if (!g->pass && dist < NAV_POS_TOL_MM * 0.5f) v = 0;

    /* 角速度 */
    float wmax = DEG2RAD(NAV_WMAX_DPS);
    float w = NAV_KP_ANG * eth;
    float w_brake = sqrtf(2.0f * DEG2RAD(NAV_WACC_DPS2) * 0.8f * fabsf(eth));
    if (fabsf(w) > w_brake) w = copysignf(w_brake, eth);
    w = clampf(w, -wmax, wmax);
    if (fabsf(eth) > DEG2RAD(NAV_ANG_TOL_DEG) * 0.5f && fabsf(w) < MIN_W_RADPS) w = copysignf(MIN_W
_RADPS, eth);
    if (!g->pass && fabsf(eth) < DEG2RAD(NAV_ANG_TOL_DEG) * 0.5f) w = 0;
```

```
    /* 场地方向 → 车体方向 */
    float vx_f = dist > 1e-3f ? v * ex / dist : 0.0f;
    float vy_f = dist > 1e-3f ? v * ey / dist : 0.0f;
    float c = cosf(p.th), s = sinf(p.th);
    chassis_set_velocity(c * vx_f + s * vy_f, -s * vx_f + c * vy_f, w);
}
```

这是整个走位的核心，逻辑分四步：

1. **算误差：**离目标的距离 dist、角度差 eth。
2. **判断到没到：**"途经点"只要进入 40mm（急转弯 15mm）就转向下一段，不停车；"终点"要位置差 < 6mm 且角度差 < 1.5°，并且**连续 3 个周期**都满足才算停稳（防止冲过头时恰好经过一下就算到了）。
3. **算速度大小：**取三者最小——最大速度 vmax；"刹车速度" $\sqrt{(v_exit^2 + 2 \cdot a \cdot 0.8 \cdot dist)}$；比例项 KP × dist。
4. **换方向：**速度方向在场地坐标里是指向目标的，要旋转到车体坐标再交给底盘。

刹车速度的含义：以加速度 a 减速，从速度 v 刹停需要 $v^2/(2a)$ 的距离。反过来，离目标还有 dist 时，速度最多只能是 $\sqrt{(2 \cdot a \cdot dist)}$，否则来不及停。乘 0.8 是留余量。举例：离终点 100mm 时，$\sqrt{(2 \times 700 \times 0.8 \times 100)} \approx 335$mm/s，比例项 $3 \times 100 = 300$mm/s，取 300；离 20mm 时比例项只有 60mm/s，车平稳地减速靠近。

9.2 后台执行：nav_start_route 和 nav_wait

一次走位被拆成若干"段"（seg），存在 `s_seg[]` 数组里，`nav_update()` 由 sys_poll 每 20ms 调用一次，按顺序执行。所以有两种用法：

两种用法

```
nav_route_to(目标);             /* 用法1：一直等到车到位才返回（简单） */

nav_start_route(目标);          /* 用法2：发令后立刻返回，车在后台开 */
arm_pick_from_slot(i);           /*         这期间机械臂做别的事 */
nav_wait();                      /*         最后等车到位 */
```

用法 2 节省了大量时间：在同一个加工区内从一个圆环平移到下一个圆环（约 1.5 秒），同时机械臂从载物槽取下一个物料（约 2.5 秒），两件事重叠进行。

9.3 路线规划：plan_route

nav.c · plan_route()

```
/*
 * 路线规划：
 *  1. 车道网格上找最短路：起点 → 节点… → 终点
 *  2. 找一个"能安全原地转身"的点（车身转一圈扫过的范围不压线），在那里停下转到最终朝向；
 *     其余路段都保持朝向不变（麦轮可以横着走、斜着走，不需要边走边转）
 *  3. 能直接连过去的点就省掉
 */
static int plan_route(pose_t from, pose_t target, wp_t wp[MAX_WP])
{
    pose_t raw[MAX_WP];
    int n = 0;
    raw[n++] = from;
    pose_t from_same = from;  from_same.th = target.th;
```

```
    bool direct_ok = rotation_safe(from.x, from.y, from.th, target.th) ? segment_safe(from_same, ta
rget)
                                                                       : segment_safe(from, targe
t);
    if (!direct_ok) {
        node_id_t path[NODE_COUNT];
        point_t fp = { from.x, from.y }, tp = { target.x, target.y };
        int k = shortest_path(field_nearest_node(fp), field_nearest_node(tp), path);
        if (k == 0) return 0;
        for (int i = 0; i < k; i++) { point_t q = field_node(path[i]); pose_t w = { q.x, q.y, 0 };
raw[n++] = w; }
    }
    raw[n++] = target;

    /* 找转身点：第一个能安全原地转身的点；不需要转身时 k_rot = 0 */
    int k_rot = -1;
    bool need_rot = fabsf(wrap_pi(target.th - from.th)) > DEG2RAD(3.0f);
    if (!need_rot) k_rot = 0;
    for (int i = 0; i < n && k_rot < 0; i++)
        if (rotation_safe(raw[i].x, raw[i].y, from.th, target.th)) k_rot = i;
    if (k_rot < 0) { LOG("nav: 路线上没有能安全转身的地方，只能在终点转"); k_rot = n - 1; }
    for (int i = 0; i < n; i++) raw[i].th = (i < k_rot) ? from.th : target.th;

    /* 如果从转身点到下一个点"边走边转"整段都安全，就不用停车转身（省 1~2 秒） */
    bool rot_moving = false;
    if (need_rot && k_rot < n - 1) {
        pose_t a = raw[k_rot]; a.th = from.th;
        if (segment_safe(a, raw[k_rot + 1])) { rot_moving = true; raw[k_rot].th = from.th; }
    }

    /* 贪心省点：在转身点前后两段里，分别尽量直接连到最远的安全点（不能跨过转身点） */
    int m = 0;
    wp[m].p = from; wp[m].rotate_here = (need_rot && !rot_moving && k_rot == 0); m++;
    int i = 0;
    while (i < n - 1) {
        int lim = (i < k_rot) ? k_rot : n - 1;
        int j = lim;
        while (j > i + 1 && !segment_safe(raw[i], raw[j])) j--;
        wp[m].p = raw[j];
        wp[m].rotate_here = (need_rot && !rot_moving && j == k_rot && k_rot > 0);
        if (wp[m].rotate_here) wp[m].p.th = from.th;     /* 到这个点时还是原来的朝向，停下后再转 */
        m++;
        i = j;
    }
    return m;
}
```

步骤：

1. **能直接开过去吗？** 用 `segment_safe` 检查从起点到终点的直线：每 25mm 取一个位置，检查车身是否压线。安全就直接开。
2. **不能就走车道网格：** 找离起点、终点最近的节点，用 **Dijkstra 最短路算法**在 9 个节点里找最短路线（`shortest_path`）。节点只有 9 个，算一次不到 1ms。被标记"有障碍"的路段会被跳过。
3. **在哪里转身？** 车转身时车角会扫出一个半径 189mm 的圆（$\sqrt{(117^2+148^2)}$），在窄处转身会压线。`rotation_safe` 检查在某点原地转身是否安全， 程序选路线上第一个安全的点转身，其余路段保持朝向不变——麦轮可以横着走、斜着走，并不需要"车头对着前进方向"。 如果从转身点到下一个点"边走边转"整段都安全，就不停车，边走边转（省 1~2 秒）。
4. **省掉多余的点：** 如果从第 i 个点能直接安全地开到第 j 个点，中间的点就省掉。

以图 8-1 的轨迹为例：从转盘去粗加工区，车在 TC 节点（上中）原地转身（车头从朝上转到朝下），然后沿中间竖道一路开到 BC，再横移到圆环前。

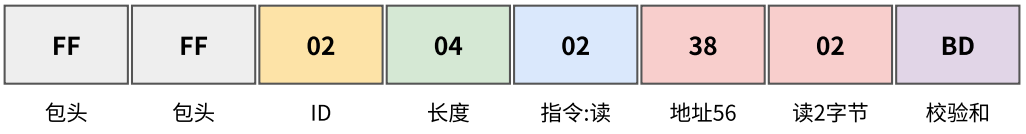
9.4 两个"从模拟中学到"的细节

- **预备点：**目标贴着禁行区（加工区、转盘前）时，先规划到目标后方 90mm 的"预备点"、把朝向摆正，最后一段只直线前进。 否则车可能在最后一段边走边转，车角扫进加工区。
- **急转弯减速：**路线在途经点处拐弯超过 45° 时，过点速度降到 50mm/s。模拟中发现，带着 120mm/s 的横向速度拐 80° 的弯，车会甩出约 50mm，刚好碰到禁行区角。

10　舵机通信 bus_servo：飞特协议逐字节拆解

ST3215 和 SC09 都是飞特（Feetech）的总线舵机：多个舵机串在一根线上，每个有自己的 ID。单片机串口通过舵机驱动板（Bus Servo Adapter A）连到总线。 通信是"一问一答"：单片机发一个数据包，被点名的舵机回一个数据包。

例：读 2 号舵机（肩）的当前位置 —— 单片机发出 8 个字节



长度 = 参数个数(2) + 2 = 4

校验和：0x02+0x04+0x02+0x38+0x02 = 0x42，按位取反 → 0xBD

图 10-1　读 2 号舵机当前位置的数据包

bus_servo.c · 组包发送

```
static void send_packet(port_uart_t uart, uint8_t id, uint8_t inst, const uint8_t *prm, uint8_t n)
{
    uint8_t b[64];
    uint8_t len = (uint8_t)(n + 2);
    uint8_t sum = (uint8_t)(id + len + inst);
    b[0] = 0xFF; b[1] = 0xFF; b[2] = id; b[3] = len; b[4] = inst;
    for (uint8_t i = 0; i < n; i++) { b[5 + i] = prm[i]; sum = (uint8_t)(sum + prm[i]); }
    b[5 + n] = (uint8_t)~sum;
    port_uart_flush_rx(uart);
    port_uart_write(uart, b, (uint16_t)(n + 6));
    memcpy(s_last_tx, b, (size_t)(n + 6));
    s_last_tx_len = (uint16_t)(n + 6);
}
```

指令	编号	用途
PING	0x01	点名：舵机在不在（`sping` 命令用它）
READ	0x02	读寄存器：当前位置（56）、是否在运动（66）
WRITE	0x03	写寄存器：目标位置（42）、力矩开关（40）、ID（5）
SYNC WRITE	0x83	一包同时给多个舵机下命令，三个关节同时出发

10.1 ST3215 和 SC09 的两个区别

	ST3215（STS 系列）	SC09（SCS 系列）
位置范围	0~4095 = 360°（每度约 11.4）	0~1023 = 300°
两字节数据	**低字节在前**（小端）	**高字节在前**（大端）
电压	6~12.6V（接 3S 支路）	**4~6V（只能接 6V 降压，接 12V 会烧）**

bus_servo.c · 按系列决定字节顺序

```
static void put16(uint8_t *d, int v, servo_proto_t proto)
{
    uint16_t u = (uint16_t)v;
    if (proto == SERVO_STS) { d[0] = (uint8_t)(u & 0xFF); d[1] = (uint8_t)(u >> 8); }
    else                    { d[0] = (uint8_t)(u >> 8);   d[1] = (uint8_t)(u & 0xFF); }
}
```

10.2 回声问题

舵机总线是"单线半双工"：发和收共用一根线。有些驱动板会把我们发出去的数据也送回单片机的 RX（"回声"）。`read_status()` 收到一个完整包后，先和刚发出去的包比对，一模一样就跳过，再继续等真正的回复。等回复最多 4ms，超时返回 −1。

写命令（WRITE、SYNC WRITE）不等回复，节省时间；下次读之前会清空接收缓冲区，所以残留的回复不会影响下一次。

11　机械臂 arm：逆运动学与取放动作

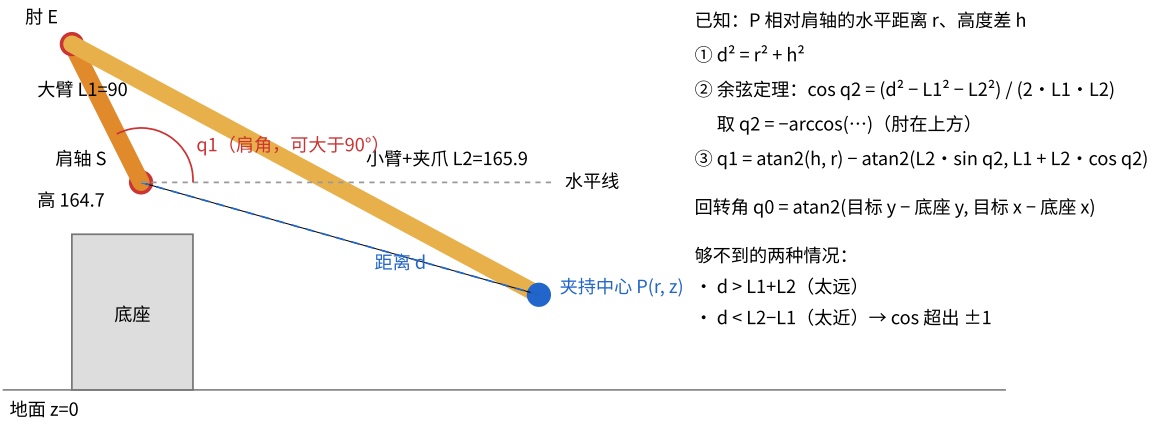


图 11-1　机械臂侧视图与逆运动学公式

机械臂有 3 个关节决定夹爪位置：**回转**（绕竖直轴转，决定朝哪个方向）、**肩**（大臂抬起的角度）、**肘**（小臂相对大臂的角度）。腕部是固定的。 我们希望程序这样用：`arm_move_to(目标点 x, y, z)`，由程序自己算三个关节角——这叫**逆运动学（IK）**。

arm.c · arm_ik()

```
bool arm_ik(vec3_t p, float q[3])
{
    float dx = p.x - ARM_BASE_X_MM, dy = p.y - ARM_BASE_Y_MM;
    float c = hypotf(dx, dy);                                   /* 回转轴到目标的水平距离 */
    if (c <= fabsf(ARM_TOOL_SIDE_MM)) return false;
    float reach = sqrtf(c * c - ARM_TOOL_SIDE_MM * ARM_TOOL_SIDE_MM);    /* 摆动平面内的水平伸出距离 */
    q[0] = atan2f(dy, dx) - atan2f(ARM_TOOL_SIDE_MM, reach); /* 回转：夹爪侧偏 16mm，所以要比"正对目
标"少转一点 */
    if (q[0] < QMIN[0]) q[0] += 2.0f * PI_F;                    /* 换算到回转的工作范围内（例如 -177° → 18
3°） */
    if (q[0] > QMAX[0]) q[0] -= 2.0f * PI_F;
    float r = reach - ARM_SHOULDER_R0_MM;                       /* 肩轴到目标的水平距离 */
    float h = p.z - ARM_SHOULDER_Z_MM;                          /* 肩轴到目标的竖直距离 */
    float d2 = r * r + h * h;
    /* 余弦定理求肘角 */
    float c3 = (d2 - ARM_L1_MM * ARM_L1_MM - ARM_L2_MM * ARM_L2_MM) / (2.0f * ARM_L1_MM * ARM_L2_M
M);
    if (c3 > 1.0f || c3 < -1.0f) return false;                  /* 太远或太近，够不到 */
    q[2] = -acosf(c3);                                          /* 取"肘部朝上"的解（和动画一致） */
    q[1] = atan2f(h, r) - atan2f(ARM_L2_MM * sinf(q[2]), ARM_L1_MM + ARM_L2_MM * cosf(q[2]));
    for (int j = 0; j < 3; j++) {
        if (q[j] < QMIN[j] || q[j] > QMAX[j]) return false;  /* 超出软限位 */
        int raw = q_to_raw(j, q[j]);
        if (raw < 0 || raw > 4095) return false;             /* 超出舵机行程：ZERO 标定不合理 */
    }
    return true;
}
```

11.1 一步步算

1. **回转角：**俯视看，目标在回转轴的哪个方向是 `atan2(dy, dx)`。但按 CAD，夹持中心不在大臂/小臂的摆动平面里，而是偏到左侧 **16mm**（`ARM_TOOL_SIDE_MM`），所以回转要比"正对目标"少转一个小角度 `atan2(16, 伸出距离)`，伸出距离 = √(目标水平距离² − 16²)。
2. 转过去以后，问题变成**竖直平面里的两连杆**：肩轴到目标的水平距离 r、高度差 h，两根杆长 L1 = 90、L2 = 165.9。注意肩轴在回转轴**后面** 10.4mm（`ARM_SHOULDER_R0_MM = -10.4`），所以 r = 伸出距离 + 10.4。
3. **肘角：**肩、肘、目标三点组成三角形，三边长 L1、L2、d 都已知，用余弦定理求夹角。有两个解（肘朝上、肘朝下），取"肘朝上"——肘朝下会撞到车顶板。
4. **肩角：**目标方向角 atan2(h, r) 减去"小臂造成的偏角"。
5. **检查软限位：**超出 robot_config.h 里设定的关节范围就返回 false（够不到）。

11.2 三个实际例子（用同一公式算的）

目标	车体坐标 (x, y, z)	r, h（mm）	回转 / 肩 / 肘
转盘取料点（夹持中心）	(197, −45, 146)	162, -19	-6° / 70° / -108°
粗加工区放置点	(217, −45, 57)	182, -108	-5° / 18° / -73°
载物槽 0 上方（安全高度）	(−50, −50, 235)	104, 70	173° / 133° / -132°

机械结构要注意：小臂+夹爪（166mm）比大臂（90mm）长很多，所以够车上近处的载物槽时，大臂要向后仰到约 133°，小臂向前下折到约 -132°。请在实物上确认肩、肘能转到这么大的角度而不互相干涉。如果做不到，可以考虑把载物槽挪远一点，或缩短小臂。

11.3 关节角 → 舵机数值

arm.c

```
static int q_to_raw(int j, float q) { return ZERO[j] + (int)lroundf((float)DIR[j] * q * ST_COUNTS_PER_RAD); }
```

ST3215 一圈 4096 个数，1 弧度 = 4096/(2π) ≈ 651.9 个数。`ZERO` 是"角度为 0 时舵机的读数"，`DIR` 是方向（舵机装反了就是 −1）。这两个必须用示教法标定（第 17 章）。回转关节的零点特意设在 1024：它要能转到 −80°~260°（车头方向和三个载物槽都要够到），0~4095 正好对应 −90°~270°。

11.4 取和放：每次只用 3 个动作

arm.c · arm_pick() / arm_place()

```
/*
 * 取：张爪（不等，边移动边张开）→ 到上方 → 下降 → 合爪 → 直接抬回安全高度
 * 放：到上方 → 下降 → 张爪 → 抬回安全高度
 * 每次取或放只用 3 个移动动作；结束时机械臂总在安全高度，下一步可以直接转过去。
 */
int arm_pick(vec3_t base)
{
    vec3_t g = grip_point(base);
```

```
    float q[3];
    if (!arm_ik(g, q) || !arm_ik(above_of(g), q)) { LOG("arm: 取料点够不到 (%d,%d,%d)", (int)g.x, (in
t)g.y, (int)g.z); return -2; }
    servo_write_pos(&SV[J_GRIP], GRIP_RAW_OPEN, 0, 0);
    if (arm_move_safe(g) != 0) return -2;
    bool has = false;
    arm_grip_close(&has);
    arm_move_to(above_of(g));
    if (!has) { LOG("arm: 没夹到"); servo_write_pos(&SV[J_GRIP], GRIP_RAW_OPEN, 0, 0); return -1; }
    return 0;
}

int arm_place(vec3_t base)
{
    vec3_t g = grip_point(base);
    g.z += 1.0f;                                                    /* 留 1mm，让物料自己落稳而不是被压下去 */
    float q[3];
    if (!arm_ik(g, q) || !arm_ik(above_of(g), q)) { LOG("arm: 放置点够不到 (%d,%d,%d)", (int)g.x, (in
t)g.y, (int)g.z); return -2; }
    if (arm_move_safe(g) != 0) return -2;
    arm_grip_open();
    arm_move_to(above_of(g));
    return 0;
}
```

- **安全高度 235mm：**机械臂在车上方转动时，夹持中心不低于这个高度，避免扫到屏幕和槽里的物料。所以每次都是"在高处转过去 → 垂直下降 → 夹/放 → 垂直抬起"。
- **夹持点：**物料是上 Ø30、下 Ø50、高 60 的台体，夹爪最大开口约 46mm，只能夹上面 Ø30 那段（高 8mm），夹持中心在物料底面以上 56mm。
- **放置时高 1mm：**让物料自己落稳，而不是被夹爪压下去。
- **怎么知道夹没夹到：**合爪后读 SC09 的位置。没夹到东西时夹爪会合到底（读数接近 `GRIP_RAW_EMPTY_CLOSED`）；夹到了会被物料挡住，停在更大的读数。

arm.c · move_raw()：三个关节同时出发、同时到达

```
/* 三个关节同时出发、同时到达：按最大的转动量算时间，再给每个关节分配速度 */
static int move_raw(const int target[3])
{
    int maxd = 0;
    for (int j = 0; j < 3; j++) {
        int d = target[j] - s_raw[j];
        if (d < 0) d = -d;
        if (d > maxd) maxd = d;
    }
    if (maxd < 3) return 0;
    float T = (float)maxd / (float)ARM_SPEED_RAW_PER_S;     /* 秒 */
    uint8_t ids[3] = { SERVO_ID_YAW, SERVO_ID_SHOULDER, SERVO_ID_ELBOW };
    int spd[3];
    for (int j = 0; j < 3; j++) {
        int d = target[j] - s_raw[j];
        if (d < 0) d = -d;
        spd[j] = (int)((float)d / T) + 20;
    }
    servo_sync_write_pos(PORT_UART_SERVO_ST, ids, target, spd, ARM_ACC, 3);
    for (int j = 0; j < 3; j++) s_raw[j] = target[j];
    /* 先等理论时间（含加速段），再查询舵机"是否还在动"，三个都停了才算完成 */
    float ramp = (float)ARM_SPEED_RAW_PER_S / ((float)ARM_ACC * 100.0f);
    sys_wait_ms((uint32_t)((T + ramp) * 1000.0f));
    uint32_t t0 = sys_now();
```

```
    while ((uint32_t)(sys_now() - t0) < 400) {
        bool moving = false;
        for (int j = 0; j < 3; j++) {
            uint8_t m = 0;
            if (servo_read_byte(&SV[j], SREG_MOVING, &m) == 0 && m) moving = true;
        }
        if (!moving) break;
        sys_wait_ms(10);
    }
    return 0;
}
```

三个关节的转动量不同，如果都用同一速度，会一个先到一个后到，夹爪走的是一条奇怪的弯路。这里先按转动量最大的那个关节算出总时间 T， 再给每个关节分配"转动量 ÷ T"的速度，三个关节就同时到达。下发后先等理论时间（含加速段），再查询每个舵机的"是否在运动"寄存器，三个都停了才继续—— 这样不会出现"手臂还没降到位就合爪"的情况。

12　视觉通信 vision：协议与"理想点+偏差"

摄像头型号还没定，所以单片机这边只规定"通信协议"，相机那边用什么识别算法都行，按协议发数据就能接上。 协议是文本行，每行以 `$` 开头、换行结尾，用串口助手就能直接看懂、手动模拟：

方向	格式	含义
单片机→相机	`$MODE,n`	切换识别模式：0 空闲，1 二维码，2 转盘颜色，3 圆环，4 障碍物
相机→单片机	`$QR,156+123+516+231`	识别到的任务码
相机→单片机	`$TT,颜色,稳定,dx,dy`	取料点上物料的颜色（1~6，0=没有）；转盘是否停稳；物料相对理想取料点的偏差（mm，车体坐标）
相机→单片机	`$RING,找到,dx,dy[,dth]`	圆环相对理想放置点的偏差；可选 dth：由区域边线算出的车头朝向误差（度）
相机→单片机	`$OBS,距离`	前方障碍物距离（预留）

行尾可以加 `*XX` 校验（NMEA 格式），有就校验，没有就不校验，方便调试时手打。

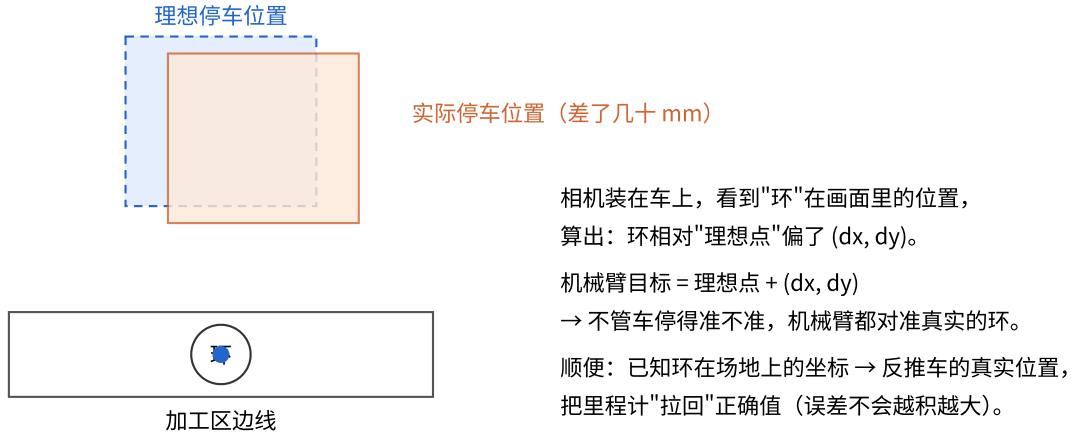


图 12-1　理想点 + 偏差：不依赖车停得准不准

mission.c · ring_target()

```
/*
 * 车到位后，返回圆环中心的车体坐标（给机械臂用）。
 * 相机看到圆环时：用"理想点+偏差"，并顺便用这个已知位置的圆环把里程计校正回来，
 * 这样走得越远误差也不会越积越大。相机看不到时：只能按里程计算。
 */
static vec3_t ring_target(place_zone_t z, int ring, float surface_z)
{
    pose_t pp = field_place_pose(z, ring);
    point_t rf = field_ring_point(z, ring);
    nav_wait();
    vision_ring_t vr;
    uint32_t t0 = sys_now();
    while ((uint32_t)(sys_now() - t0) < 150) sys_poll();      /* 等相机拿到停稳后的新画面 */
    if (vision_get_ring(&vr, 120) && vr.found && fabsf(vr.dx) < 75.0f && fabsf(vr.dy) < 75.0f) {
        vec3_t p = ideal_point(pp, rf, surface_z);
```

```
        p.x += vr.dx;
        p.y += vr.dy;
        /*
         * 里程计校正：圆环在车体坐标 (p.x,p.y)、在场地坐标 rf → 反推车的真实位置。
         * 相机还给了朝向误差 dth 时，连朝向一起校正（否则沿用陀螺仪/里程计的朝向）。
         */
        pose_t cur = chassis_pose();
        float th = vr.has_dth ? wrap_pi(pp.th + DEG2RAD(vr.dth_deg)) : cur.th;
        float c = cosf(th), s = sinf(th);
        pose_t fixed = { rf.x - (c * p.x - s * p.y), rf.y - (s * p.x + c * p.y), th };
        chassis_set_pose(fixed);
        return p;
    }
    return to_arm(rf, surface_z);
}
```

这是全程精度的关键。里程计（轮子算出来的位置）走得越远误差越大，几米下来可能差几十毫米；但相机每到一个圆环都"看一眼"， 直接测出圆环相对机械臂的位置——**放置精度只取决于相机标定，和车停得准不准无关**。同时用这个已知位置的圆环把里程计校正回来， 下一段路就从准确的位置出发。模拟中，打滑 2% 的情况下放置偏差仍 ≤1.5mm（而规则 1 环要求 ≤1.5mm 才得 15 分）。

13　任务码 task_code：156+123+516+231

组	数字	含义	解析结果
第 1 组	156	第一批颜色和顺序	红(1) → 黑(5) → 浅蓝(6)
第 2 组	123	第一批放在粗加工区**和**暂存区的圆环	红→1 号，黑→2 号，浅蓝→3 号
第 3 组	516	第二批颜色和顺序	黑 → 红 → 浅蓝
第 4 组	231	第二批放在**粗加工区**的圆环	黑→2 号，红→3 号，浅蓝→1 号
（推出）	—	第二批在**暂存区**：叠到第一批同颜色上	黑→2 号（第一批黑在 2 号），红→1 号，浅蓝→3 号

> 常见误解：第二批在暂存区**不是**用第 4 组的 231，而是找第一批同颜色物料所在的环叠上去。程序里的 `stash_ring[1][]` 就是这样推出来的，单元测试 `test_task.c` 专门验证了规则原文的这个例子。

task_code.c · 推出第二批的暂存位置

```
    /* 两批必须是同样的三种颜色 */
    for (int i = 0; i < 3; i++) {
        bool found = false;
        for (int j = 0; j < 3; j++) if (t->color[1][i] == t->color[0][j]) { found = true; t->stash_
ring[1][i] = t->rough_ring[0][j]; }
        if (!found) { e = "两批颜色不一致"; goto fail; }
        t->stash_ring[0][i] = t->rough_ring[0][i];
    }
```

`task_parse()` 还会检查各种错误：不是 4 组、某组不是 3 位、颜色超出 1~6、同一批颜色重复、位置不是 1/2/3 的排列、两批颜色不一致。 相机偶尔读错一位时，程序会拒绝这个结果、继续读，而不是带着错误的任务开始搬运。

14　整场流程 mission：逐段对照比赛规则

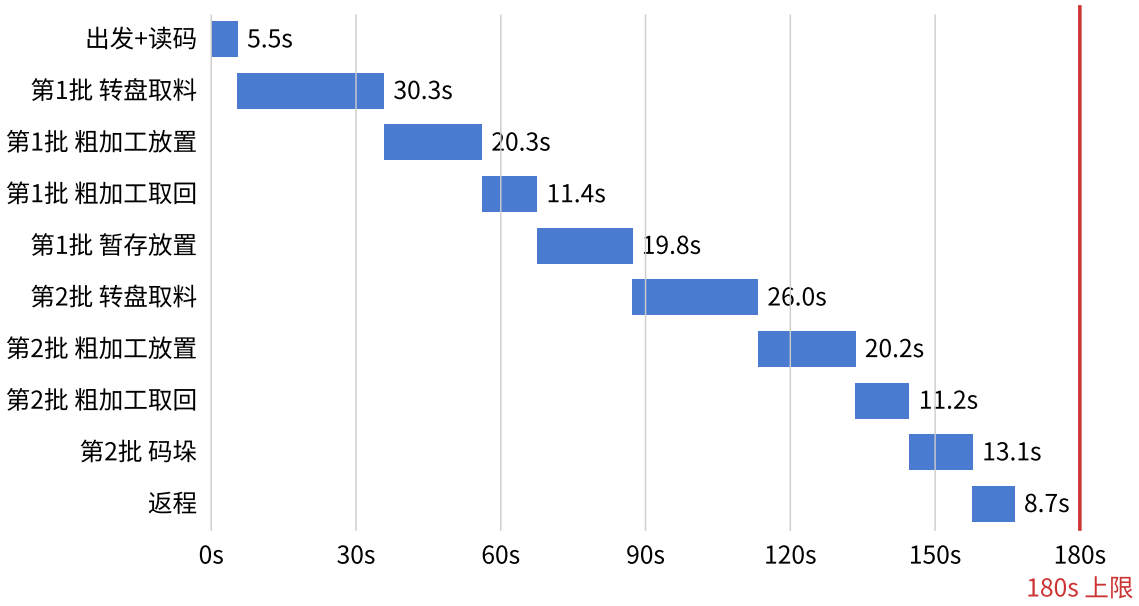


图 14-1　模拟器里一整场各阶段用时（任务码 156+123+516+231，启停区 1，总用时 166.5s）

14.1 读二维码

mission.c · read_qr()

```
/* ---------------- 1. 读二维码 ---------------- */
static bool read_qr(void)
{
    char buf[24];
    const char *err = NULL;
    vision_set_mode(VMODE_QR);
    nav_start_route(field_qr_pose());
    /* 规则：出发后方可自行展开 —— 等车身离开启停区再展开机械臂 */
    pose_t sp = field_start_pose(s_zone);
    while (nav_busy()) {
        pose_t p = chassis_pose();
        if (dist2d(p.x, p.y, sp.x, sp.y) > 350.0f) break;
        sys_poll();
    }
    arm_home();
    nav_wait();

    /* 二维码板位置随机：先原地看，看不到就沿板左右平移搜索 */
    static const float SEARCH_DY[] = { 0, 120, -120, 240, -240, 0 };
    for (unsigned k = 0; k < sizeof SEARCH_DY / sizeof SEARCH_DY[0]; k++) {
        if (k > 0) {
            pose_t p = field_qr_pose();
            p.y += SEARCH_DY[k];
            nav_goto(p, 250);
        }
        uint32_t t0 = sys_now();
        while ((uint32_t)(sys_now() - t0) < QR_TIMEOUT_MS / 3) {
            sys_poll();
            if (vision_get_qr(buf, sizeof buf, 500) && task_parse(buf, &T, &err)) {
                LOG("mission: 任务码 %s", T.text);
                return true;
```

```
            }
            if (sys_aborted()) return false;
        }
        if (err) LOG("mission: 收到的任务码无效: %s", err);
    }
    return false;
}
```

- 规则"出发后方可自行展开"：车先开，等车身离出发点 350mm（已出启停区）再展开机械臂。
- 二维码板位置"在规定范围内随机"：先原地看 2 秒，看不到就沿板上下平移 ±120、±240mm 搜索。
- 读到的字符串必须通过 `task_parse` 校验才算数。

14.2 转盘取料

mission.c · turntable_batch()

```
/* ---------------- 2. 转盘取料 ---------------- */
static void turntable_batch(int b)
{
    if (!time_ok(STAGE_EST_MS)) return;
    display_set_status(b == 0 ? "B1 RAW" : "B2 RAW");
    stage(b == 0 ? "第1批 转盘取料" : "第2批 转盘取料");
    vision_set_mode(VMODE_TURNTABLE);
    drive_to_zone(field_turntable_pose());

    for (int i = 0; i < 3; i++) {
        s_have[i] = false;
        int want = T.color[b][i];
        if (!time_ok(6000)) return;
        LOG("mission: 第%d批第%d个，等%s色转到取料点", b + 1, i + 1, color_name(want));

        for (int attempt = 0; attempt < 2 && !s_have[i]; attempt++) {
            /* 等目标颜色转过来并停稳 */
            vision_tt_t tt;
            bool seen = false;
            uint32_t t0 = sys_now();
            while ((uint32_t)(sys_now() - t0) < TURNTABLE_WAIT_MAX_MS) {
                sys_poll();
                if (sys_aborted()) return;
                if (vision_get_tt(&tt, 300) && tt.color == want && tt.stable) { seen = true; break;
}
            }
            if (!seen) { LOG("mission: 等不到%s色", color_name(want)); break; }

            /* 取料点 = 理想取料点 + 相机测得的偏差（不依赖里程计误差） */
            vec3_t p = ideal_point(field_turntable_pose(), field_turntable_pick_point(), TURNTABLE_
TOP_Z_MM);
            p.x += tt.dx;
            p.y += tt.dy;
            int r = arm_pick(p);
            if (r == 0) {
                R->grabs++;
                update_counts();
                s_have[i] = (arm_place_to_slot(i) == 0);   /* 先放到车上，再抓下一个 */
                if (!s_have[i]) LOG("mission: 放进载物槽%d失败", i);
            } else {
                LOG("mission: 抓取失败(%d)，%s", r, attempt == 0 ? "再试一次" : "跳过");
            }
        }
    }
```

```
    vision_set_mode(VMODE_IDLE);
}
```

- 按任务码顺序，一个一个等：相机报告"取料点上是我要的颜色，并且转盘停稳了"才抓。最多等 20 秒（规则：停滞 15s 结束，等转盘另加 8s）。
- 取料点 = 理想取料点 + 相机偏差（第 12 章）。
- 抓到后**立刻放进第 i 个载物槽**，再抓下一个——这正是规则要求的"每次抓取 1 个，放到车上后才能抓取下一个"。
- 没夹到就再试一次（转盘下一次转过来时）。

14.3 加工区放置：place_all（粗加工区、暂存区共用）

mission.c · place_all()

```
/*
 * 把车上的物料按顺序放到某个区域的圆环上（粗加工区放置、暂存区放置/码垛共用）
 * 第一个物料：先开到区域、停稳，再从载物槽取（跨区时物料必须在槽里）；
 * 之后的物料：车在区域内平移到下一个圆环的同时，机械臂从载物槽取料。
 */
static void place_all(int b, place_zone_t z, const int rings[3], bool stacking)
{
    bool first = true;
    for (int i = 0; i < 3; i++) {
        if (!s_have[i]) continue;
        int ring = rings[i];
        if (stacking && !s_base_ok[ring]) {
            LOG("mission: %d号环没有第一批同色物料，第二批%s色不能码垛，留在车上", ring, color_name(T.color
[1][i]));
            continue;
        }
        if (!time_ok(5000)) return;
        if (first) { drive_to_zone(field_place_pose(z, ring)); first = false; }
        else       nav_start_route(field_place_pose(z, ring));
        int r = arm_pick_from_slot(i);                    /* 与平移同时进行 */
        s_have[i] = false;
        if (r != 0) { nav_wait(); LOG("mission: 从载物槽%d取料失败", i); continue; }
        float surface = stacking ? MAT_HEIGHT_MM : FLOOR_Z_MM;   /* 码垛：叠在第一批顶上 */
        vec3_t p = ring_target(z, ring, surface);
        if (arm_place(p) == 0) {
            R->places++;
            if (z == ZONE_ROUGH) s_on_rough[i] = true;
            else if (b == 0) s_base_ok[ring] = true;
            else R->stacks++;
            update_counts();
        }
    }
}
```

注意 `first` 变量：第一个物料用 `drive_to_zone`——先开到区域、停稳，再从载物槽取料（跨区行驶时物料在槽里，守规则）；之后的物料用 `nav_start_route` 后台平移，**同时**机械臂从载物槽取料（同一区域内，规则不限制）。码垛时 surface = 60mm（叠在第一批顶上），并且检查对应的第一批物料是否确实放好了，没放好就不叠（叠了也不得分，还可能倒）。

14.4 粗加工区取回

mission.c · rough_retrieve()

```
/* ---------------- 4. 粗加工区取回 ---------------- */
static void rough_retrieve(int b)
{
    stage(b == 0 ? "第1批 粗加工取回" : "第2批 粗加工取回");
    int pending = -1;                                   /* 夹在手里、还没放进载物槽的物料 */
    for (int i = 0; i < 3; i++) {
        if (!s_on_rough[i]) continue;
        if (!time_ok(5000)) break;
        int ring = T.rough_ring[b][i];
        nav_start_route(field_place_pose(ZONE_ROUGH, ring));
        if (pending >= 0) {                             /* 边平移边把上一个放进载物槽 */
            s_have[pending] = (arm_place_to_slot(pending) == 0);
            pending = -1;
        }
        vec3_t p = ring_target(ZONE_ROUGH, ring, FLOOR_Z_MM);
        if (arm_pick(p) == 0) {
            R->grabs++;
            update_counts();
            pending = i;
        }
        s_on_rough[i] = false;
    }
    if (pending >= 0) s_have[pending] = (arm_place_to_slot(pending) == 0);   /* 离开前必须放回车上 */
    vision_set_mode(VMODE_IDLE);
}
```

用一个 `pending` 变量记录"夹在手里还没放进槽的物料"：开往下一个圆环的同时把它放进载物槽。离开区域前一定把最后一个放回车上。

14.5 时间管理 time_ok

mission.c · time_ok()

```
/* 剩余时间够不够再做一个耗时 need_ms 的动作，并且做完还能回家？ */
static bool time_ok(uint32_t need_ms)
{
    uint32_t el = sys_match_elapsed();
    uint32_t back = nav_estimate_route_ms(field_start_pose(s_zone));
    bool ok = el + need_ms + back + MATCH_RETURN_RESERVE_MS < MATCH_TIME_LIMIT_MS;
    if (!ok && !R->aborted_by_time) {
        LOG("mission: 时间不够了（已用 %lus，返程约 %lus），放弃剩余任务直接回家",
            (unsigned long)(el / 1000), (unsigned long)(back / 1000));
        R->aborted_by_time = true;
    }
    return ok;
}
```

每做一个动作前都问一下：**已用时间 + 这个动作要的时间 + 从当前位置回启停区的时间 + 余量 6 秒 < 180 秒吗？**不够就放弃剩下的任务，直接回家。因为返程有 2 分、两项统计各 2 分，而且不回启停区整场可能不完整；最后一个码垛（最多 15 分）来不及做，比冒险超时划算。举例：已用 145s、码垛一个约 7s、返程约 11s，145+7+11+6 = 169 < 180，可以做。

返程时间由 `nav_estimate_route_ms()` 按规划路线估算，系数是用模拟器实测对出来的（估计值约比实际多 1 秒）。

15　屏幕、陀螺仪、调试命令

15.1 屏幕 display

规则要求：屏幕在车体上部醒目处、发光、不被遮挡，**字高 ⩾ 12mm**，显示全部任务码和完成情况。程序显示 4 行：

`156+123+`
`516+231`
`G12 P12`（抓取、放置正确次数）
`DONE`（状态）

每行最多 8 个字符，方便用大字号。2.42 寸 OLED 在 12mm 字高下只能放约 2 行×7 字符，放不下；建议 3.5 寸 480×320 屏。 屏幕型号确定后，只要写一个"后端"——3 个函数 init / clear / text(行号, 字符串)——用 `display_set_backend()` 挂上，上层代码不用改。

display.h · 屏幕后端接口

```
typedef struct {
    void (*init)(void);
    void (*clear)(void);
    void (*text)(int line, const char *s);   /* 在第 line 行（0~3）显示一串 ASCII 字符 */
} display_backend_t;
```

15.2 陀螺仪 MPU6050（GY-521 模块）

麦轮打滑，尤其横移和原地转身时。只靠轮子算朝向，转几次身就偏几度；朝向偏 3°，车走 1 米就横向偏 52mm。陀螺仪直接测转了多少度，不受打滑影响。 模拟结果：轮子打滑 2%，**没有陀螺仪时转几次身就迷路**；装了陀螺仪后全程不压线。

原理很简单：陀螺仪测的是"角速度"（每秒转几度），每 20ms 读一次，乘以时间累加就是角度。上电时车静止 1 秒，测出"零漂"（静止时的读数），之后每次减掉。 接线：VCC→开发板 3.3V、GND→GND、SCL→PB8、SDA→PB9；XDA、XCL、AD0、INT 不接（地址 0x68）。`USE_MPU6050` 默认就是 1，没接模块时程序会自动退回只用轮子。标定见第 17 章第 4.5 步。板载的 ST480MC 磁力计（电子罗盘）离电机和电池太近，会被干扰，程序没用它。

15.3 串口调试命令 console

电脑串口助手 115200，勾选"发送新行"，输入命令回车。命令在 sys_poll 里只"收"，回到主循环才"执行"；唯一例外是急停 `s`，收到立即生效。

命令	作用
`i2c`	扫描 I2C 总线
`bat`	读电池电压
`mot <1-4> <速度>`	单独转某个通道 2 秒（速度 -45~45，脉冲/10ms）

`wheel`	依次让 左前/右前/左后/右后 向前转，检查接线与方向
`enc`	显示四个轮子累计脉冲
`move <前mm> <左mm> <转角度>`	相对移动（转角在 ±180° 内）
`spin <圈数>`	原地逆时针转 N 圈（标定 ROT_SCALE 用）
`imu [0|off|on]`	陀螺仪读数；0 累计角度清零；off/on 临时关闭/打开
`goto <x> <y> <角度>`	走到场地坐标；route 同参数但沿车道走
`pose <x> <y> <角度>`	设定当前位置；odom 显示当前位置
`sping`	检查 4 个舵机；sread 读舵机原始位置
`storque <0/1>`	舵机卸力/上力（卸力后可用手掰，配合 sread 示教）
`sraw <关节0-3> <值>` 单独转一个舵机（0回转 1肩 2肘 3夹爪）	
`sid <st|sc> <旧ID> <新ID>`	改舵机 ID（总线上只接一个）
`arm <x> <y> <z>`	机械臂夹持中心移动到车体坐标；ik 同参数只计算不动
`grip <o|c>`	夹爪开/合；fold 折叠；home 行车姿态
`slot <0-2> <p|t>`	放进(p)/取出(t)第 k 个载物槽
`qr / tt`	显示相机最近一次二维码/转盘数据
`task <任务码> [区]`	跳过读码，用给定任务码跑全程（调试用）
`run [区]`	跑全程（与按 KEY0 相同）
`stop 或 s`	急停

16　电脑模拟器

模拟器在 `sim/` 目录，**只在电脑上编译运行，不会下载到单片机**。它用 `port_sim.c` 代替 `port_stm32.c`，单片机上的所有比赛代码原封不动地在电脑上运行；`world.c` 扮演"真实世界"：

模拟的东西	怎么模拟
驱动板和电机	收到速度命令后，轮速按 60ms 时间常数逐渐跟上；累计编码器脉冲；可设打滑比例和随机噪声
车的真实位置	由轮速按麦轮运动学积分（与程序里的里程计分开，所以能看出里程计误差）
舵机	解析真实的飞特数据包；按速度和加速度（梯形曲线）移动；回答位置、"是否在运动"
夹爪与物料	合爪时，夹持点 9mm 内有物料就夹住；松爪时判断落在载物槽、地面，还是叠在另一个物料上（偏差 >12mm 判定倒了）
转盘	8 秒一圈，方向随机；物料转到正对场地时停 2.5 秒；第一批取完 4 秒后补上第二批（顺序随机）
相机	车停在合适位置时，按协议发出二维码、转盘颜色和偏差、圆环偏差
裁判	每 20ms 检查车身是否压线；结束时按评分表估算得分

16.1 怎么运行

命令行

```
cd sim
make                                # 编译（Windows 用 MSYS2 或 WSL 的 gcc）
./test_task                         # 任务码单元测试
./sim                               # 跑一整场，打印完整日志
./sim 156+123+516+231 2             # 指定任务码、启停区
./sim 156+123+516+231 1 2 1         # 打滑 2%，装陀螺仪
./sim --batch                       # 批量测试 + 汇总
```

16.2 模拟结果（参数标定正确的前提下）

任务码	启停区	用时	抓/放/码垛	压线次数	最大放置偏差	回区	估分
156+123+516+231	区1	169.8s	抓12 放11 码垛2	0	0.2mm	是	196分
156+123+516+231	区1	167.5s	抓12 放11 码垛2	0	0.2mm	是	196分
156+123+516+231	区2	169.5s	抓12 放11 码垛2	0	0.2mm	是	196分
156+123+516+231	区2	167.2s	抓12 放11 码垛2	0	0.2mm	是	196分
134+123+431+231	区1	166.9s	抓12 放11 码垛2	0	0.2mm	是	196分
134+123+431+231	区1	165.9s	抓12 放 9 码垛0	0	0.2mm	是	166分
134+123+431+231	区2	167.1s	抓12 放11 码垛2	0	0.2mm	是	196分
134+123+431+231	区2	177.5s	抓12 放10 码垛1	0	0.2mm	是	181分
256+312+625+213	区1	167.5s	抓12 放12 码垛3	0	1.5mm	是	206分
256+312+625+213	区1	168.8s	抓12 放10 码垛1	0	1.5mm	是	176分

256+312+625+213	区2	167.2s	抓12 放12 码垛3	0	1.6mm	是	206分
256+312+625+213	区2	169.0s	抓12 放10 码垛1	0	1.6mm	是	176分
421+321+142+132	区1	167.7s	抓11 放 9 码垛0	0	1.8mm	是	159分
421+321+142+132	区1	174.4s	抓12 放10 码垛1	0	1.8mm	是	176分
421+321+142+132	区2	165.8s	抓12 放 9 码垛0	0	1.6mm	否	159分
421+321+142+132	区2	174.1s	抓12 放10 码垛1	0	1.6mm	是	176分
365+231+653+312	区1	171.6s	抓12 放10 码垛1	0	1.6mm	是	176分
365+231+653+312	区1	169.4s	抓12 放12 码垛3	0	1.6mm	是	206分
365+231+653+312	区2	172.0s	抓12 放10 码垛1	0	1.5mm	是	176分
365+231+653+312	区2	169.6s	抓12 放12 码垛3	0	1.5mm	是	206分

怎么看：20 场全部不压线、全部 3 分钟内完成；12 次抓取基本都完成；放置偏差都在 3mm 以内。 时间偏紧（150~178 秒），最后 1~3 个码垛常被时间管理放弃——主要卡在等转盘转到需要的颜色。偶尔"回区：否"是因为车宽 296mm、启停区 300mm，只有 2mm 余量（第 20 章）。

> **模拟不等于实车**。模拟器假设所有参数都标定准确、相机完全准确、物料不会被碰倒。它证明的是"程序逻辑是对的"：顺序、计数、路线、时间管理都没问题。 实车能拿多少分，取决于第 17 章的标定和调试做得多好。

17　上车调试完整步骤

原则：**从下往上**，一层确认好了再上一层。每一步都有"通过标准"，不通过不要往下做。先把车架空（轮子离地），串口助手打开。

第 0 步：编译下载

1. STM32CubeIDE → File → Import → Existing Projects into Workspace → 选 SmartCar_F407 文件夹。点锤子编译，应 0 错误。
2. 本工程没有 .ioc 文件，**不要用 CubeMX 重新生成代码**，否则会覆盖 main.c。
3. BOOT0 跳线接 GND；用 DAP 下载；断电重启，**通过标准：绿灯慢闪，串口打印"==== 智能搬运小车启动 ===="**。

第 1 步：驱动板通信

操作	通过标准	不通过怎么办
`i2c`	列出 0x34	查 SCL/SDA 是否接反、驱动板是否供电、是否共地
`bat`	电压合理（3S 满电约 12600mV）	数值离谱：寄存器格式与实物版本不符，告诉我实际读数

第 2 步：轮子接线与方向

1. `mot 1 15`、`mot 2 15`、`mot 3 15`、`mot 4 15`：每次转 2 秒，记下"M1 是哪个轮、往前还是往后"。
2. 改 `robot_config.h`：`WHEEL_CH_INIT` 按"左前、右前、左后、右后"的顺序填通道号（M1=0…M4=3）；`WHEEL_SIGN_INIT` 填让轮子向前转需要的符号。
3. 重新编译下载，`wheel`：**通过标准：依次看到左前、右前、左后、右后向前转**。
4. 如果某个轮子越转越快、停不下来：编码器极性反了（驱动板以为越转越慢，拼命加速），改 `MOTOR_DRV_ENC_POLARITY`。

第 3 步：每圈脉冲数

`enc` 记读数 → 用手把左前轮向前转 10 圈 → 再 `enc`，差值 ÷ 10 = 每圈脉冲数。四个轮子都测，取平均填 `ENC_COUNTS_PER_WHEEL_REV`。

第 4 步：直行、横移、转圈（车放地上）

命令	量什么	改什么
`pose 0 0 0` 然后 `move 1000 0 0`	实际前进距离	`FWD_SCALE` = 实际 ÷ 1000
`move 0 1000 0`	实际横移距离	`STRAFE_SCALE` = 实际 ÷ 1000（常见 0.85~0.95）

`imu off` 然后 `spin 2`	实际转了多少度（先临时关掉陀螺仪，测的是轮子）	`ROT_SCALE` = 旧值 × 实际角度 ÷ 720；测完 `imu on`

通过标准：各方向误差 < 1%（1 米差不到 10mm）。每改一次都重新测。地面材质会影响打滑，最好在和比赛场地相同的地垫上标定。

第 4.5 步：陀螺仪（GY-521 / MPU6050）

上电后车静止约 1 秒，串口出现 `imu: 就绪` 才算接好。

命令	做法	通过标准 / 改什么
`i2c`	扫描总线	能看到 0x68（陀螺仪）和 0x34（驱动板）
`imu`	车静止时看读数	当前角速度 0，零漂绝对值一般 < 3°/s
`imu 0`，手把车逆时针（从上往下看）转约 90°，再 `imu`	检查方向	累计约 +90；是 −90 就把 `IMU_YAW_SIGN` 改成 −1
车侧面贴着一条胶带直线摆好，`spin 2`	车按陀螺仪转两圈停下，看车侧面和直线差几度	`IMU_SCALE` = 旧值 × 实际角度 ÷ 720

为什么要标定比例？每颗陀螺仪芯片的灵敏度有 ±3% 左右的误差，3% 在转 180° 时就是 5°，和"不装陀螺仪、轮子打滑"的误差一样大。**通过标准：**标定后连续 `spin 2` 三次，每次偏差都 < 2°。

程序的两个保护：①待机时车不动，程序会一直慢慢修正零漂（芯片刚上电时温度在变，零漂也在变），所以上电后在启停区多放一会儿反而更准；②比赛中连续 0.1 秒读不到陀螺仪（杜邦线松了），自动改用轮子算朝向，所以 `ROT_SCALE` 也要标好作为后备。

第 5 步：舵机

1. **改 ID**（一次只接一个 ST3215）：回转保持 1；肩 `sid st 1 2`；肘 `sid st 1 3`。SC09 在另一条总线，保持 1。
2. `sping`：**通过标准：4 个舵机都在。**
3. **示教零点：** `storque 0` 卸力 → 用手把机械臂摆成"大臂水平向前、小臂与大臂成一直线、整体指向车头" → `sread`，三个读数填进 `YAW_ZERO_RAW`、`SHOULDER_ZERO_RAW`、`ELBOW_ZERO_RAW`。
4. **方向：**把每个关节往"向左转 / 抬起 / 向上折"的方向掰一点，`sread`：读数变大，DIR = +1；变小，DIR = −1。
5. **夹爪：**卸力后张到能套住 Ø30 顶部 → `sread` 第 4 个数填 `GRIP_RAW_OPEN`；夹紧一个物料 → 填 `GRIP_RAW_CLOSE`（再稍微小一点，保证有夹紧力）；空着合到底 → 填 `GRIP_RAW_EMPTY_CLOSED`。
6. `storque 1`，`arm 200 -45 120`：**通过标准：用尺量，夹持中心在车头前方 155mm（200−45）、高 120mm 处，误差几毫米以内。**误差大：检查 ZERO、DIR 和臂长参数。
7. `slot 0 p` / `slot 0 t`：把物料放进/取出 0 号载物槽，三个槽都试。
8. `fold`：量整车尺寸，**必须 ⩽ 300×300×400（含所有线）**，否则改 `FOLD_*_RAW`。

第 6 步：相机（选定型号后）

相机程序按第 12 章的协议发数据。没有相机时也能测：用 USB 转 TTL 模块接 PB10/PB11，在串口助手里手动发 `$QR,156+123+516+231`、`$TT,1,1,0,0` 等行；或用 `task 156+123+516+231` 跳过读码（转盘取料仍需要 $TT 数据）。

第 7 步：场地坐标

到场地量：转盘圆心与物料半径、二维码板位置、两个加工区三个圆环中心和**编号顺序**，改 `field.c` 开头的数字。可以先用 `pose` 把车摆在已知位置，再用 `goto`、`route` 走到各个停车点检查。

第 8 步：整场

KEY1 选启停区，KEY0 启动。日志里每个阶段打印"==== 某阶段 开始（已用 xx 秒）====", 对照图 14-1 找哪里慢。任何时候输入 `s` 回车急停。

18　常见问题排查

现象	可能原因	怎么查 / 怎么改
上电没反应，绿灯不闪	BOOT0 没接地；外部晶振不起振（红灯常亮 = 进了 Error_Handler）	BOOT0 接 GND；用调试器看是否停在 Error_Handler
串口没输出	波特率不对；USB 线接错口	115200 8N1；接板上 USB_UART 口
`i2c` 扫不到 0x34	线接反、没共地、驱动板没供电	万用表量驱动板 5V；SCL 接 PB8、SDA 接 PB9
轮子转但车走歪	WHEEL_CH/SIGN 填错；某个麦轮装反（A/B 轮装错位置）	`wheel` 逐个确认；从上往下看，四个轮顶部滚子应连成"O"形（菱形）；再用 `move 0 200 0` 检查能否正左平移
轮子越转越快	编码器极性反	改 MOTOR_DRV_ENC_POLARITY
走 1 米差很多	每圈脉冲数或 SCALE 没标定	重做第 3、4 步
到点后来回抖动	KP 太大；最小速度太大	调小 NAV_KP_POS；适当放宽 NAV_POS_TOL_MM
日志"nav: 超时"	被挡住；目标不可达；标定错导致永远到不了	看打印的目标和当前位置差多少
`sping` 某个舵机无应答	ID 不对；P4 跳线没拔；TX/RX 接反；舵机驱动板没供电	只接一个舵机试 ID 1；检查跳线
机械臂"够不到"	目标超出范围或超出软限位；ZERO/DIR 标错	用 `ik x y z` 只计算不动，看算出的角度是否合理
夹爪总说"没夹到"	GRIP_RAW_EMPTY_CLOSED / THRESHOLD 不合适	空夹、夹物料分别 `grip c` 后 `sread`，看读数差多少
一直"等不到某颜色"	相机没发 $TT，或颜色编号对不上；车停的位置相机看不到取料点	串口 `tt` 看最近收到的转盘数据
跑到一半"时间不够了"	正常保护机制	看阶段用时，找慢的环节提速（第 20 章）

19　参数速查表（robot_config.h）

以下表格由程序直接从 `robot_config.h` 读取生成。标【标定】的必须实测修改。

参数	当前值	说明
1. 底盘		
`WHEEL_DIAMETER_MM`	`80.0f`	麦轮直径（80mm 麦轮）
`ENC_COUNTS_PER_WHEEL_REV`	`1496.0f`	【标定】轮子转一圈驱动板计数（暂按 11线×4倍频×减速比34 估算）
`CHASSIS_HALF_TRACK_MM`	`133.0f`	【标定】左右两轮中心距离的一半（a）
`CHASSIS_HALF_BASE_MM`	`78.0f`	【标定】前后两轮中心距离的一半（b）
`ROT_SCALE`	`1.00f`	【标定】原地转 10 圈的角度修正系数，见 README
`FWD_SCALE`	`1.00f`	【标定】前进 1000mm 实际走了多少 / 1000
`STRAFE_SCALE`	`1.00f`	【标定】横移 1000mm 实际走了多少 / 1000（麦轮横移打滑更多，常见 0.85~0.95）
`ROBOT_HALF_LEN_MM`	`117.0f`	车头-车尾方向（X）的一半
`ROBOT_HALF_WID_MM`	`148.0f`	左右方向（Y，含麦轮）的一半
`MOTOR_DRV_I2C_ADDR`	`0x34`	7 位地址
`MOTOR_DRV_TYPE`	`3`	【确认】电机类型寄存器值：0无编码器 1TT 2N20 3JGB; 25GA370 霍尔编码器先按 3 试
`MOTOR_DRV_ENC_POLARITY`	`0`	【标定】编码器极性 0/1：电机越转越快停不下来时改成另一个值
`MOTOR_SPEED_LIMIT`	`45`	速度寄存器单位是"脉冲/10ms"，限制最大绝对值
`WHEEL_CH_INIT`	`{ 0, 1, 2, 3 }`	
`WHEEL_SIGN_INIT`	`{ +1, -1, +1, -1 }`	
`USE_MPU6050`	`1`	1 = 启用陀螺仪（GY-521/MPU6050）；没接时自动退回只用轮子
`IMU_YAW_SIGN`	`(+1)`	【标定】芯片朝上安装 = +1；芯片朝下（倒装）= -1。用 imu 命令检查
`IMU_SCALE`	`1.000f`	【标定】陀螺仪比例：spin 2 后 IMU_SCALE = 旧值 × 实际角度 ÷ 720
2. 导航（走位）		
`CTRL_PERIOD_MS`	`20`	底盘控制周期：每 20ms 读一次编码器、发一次速度
`NAV_VMAX_MMPS`	`380.0f`	直线最大速度（无载理论约 534，留余量）
`NAV_ACC_MMPS2`	`700.0f`	加速度上限
`NAV_WMAX_DPS`	`150.0f`	最大角速度（度/秒）

NAV_WACC_DPS2	360.0f	角加速度上限
NAV_POS_TOL_MM	6.0f	到点位置容差
NAV_ANG_TOL_DEG	1.5f	到点角度容差
NAV_PASS_TOL_MM	40.0f	途经点（不停车）容差
NAV_LANE_MARGIN_MM	30.0f	规划路线时车身四周额外留的安全距离
NAV_TIMEOUT_MS	8000	单段走位超时
NAV_KP_POS	3.0f	位置比例系数：误差 1mm → 3mm/s
NAV_KP_ANG	3.5f	角度比例系数
3. 机械臂（3×ST3215 + 1×SC09）		
SERVO_ID_YAW	1	底座回转 ST3215
SERVO_ID_SHOULDER	2	肩 ST3215
SERVO_ID_ELBOW	3	肘 ST3215
SERVO_ID_GRIP	1	夹爪 SC09（单独一条总线，ID 可以也是 1）
ARM_BASE_X_MM	45.0f	回转轴在车体坐标中的位置：偏车头
ARM_BASE_Y_MM	-45.0f	偏车的右侧
ARM_SHOULDER_R0_MM	-10.4f	肩轴离回转轴的水平距离：负号 = 肩轴在回转轴"后面"（与小臂伸出方向相反），按 CAD 核对
ARM_TOOL_SIDE_MM	16.0f	【标定】夹持中心偏离大臂/小臂摆动平面的距离（朝小臂伸出方向的左侧为正），按 CAD 为 16mm；
ARM_SHOULDER_Z_MM	164.7f	肩轴离地高度
ARM_L1_MM	90.0f	大臂：肩轴→肘轴
ARM_L2_MM	165.9f	小臂+夹爪：肘轴→夹持中心
ST_COUNTS_PER_RAD	651.8986f	
YAW_ZERO_RAW	1024	回转的工作范围是 -80°~260°（车头方向和三个载物槽都要够到），
YAW_DIR	(+1)	
SHOULDER_ZERO_RAW	2048	
SHOULDER_DIR	(+1)	
ELBOW_ZERO_RAW	2048	
ELBOW_DIR	(+1)	
YAW_MIN_DEG	(-80.0f)	
YAW_MAX_DEG	260.0f	
SHOULDER_MIN_DEG	(-20.0f)	
SHOULDER_MAX_DEG	170.0f	

`ELBOW_MIN_DEG`	`(-170.0f)`	
`ELBOW_MAX_DEG`	`60.0f`	
`GRIP_RAW_OPEN`	`700`	张开到能套住 Ø30 顶部
`GRIP_RAW_CLOSE`	`420`	夹紧（命令值，会被物料挡住）
`GRIP_RAW_EMPTY_CLOSED`	`400`	空夹时实际能走到的位置：读回接近它说明没夹到
`GRIP_EMPTY_THRESHOLD`	`25`	读回值与空夹位置相差小于它 → 判定没夹到
`ARM_SPEED_RAW_PER_S`	`2800`	约 246°/s；ST3215 12V 空载最快约 3070（0.222s/60°）
`ARM_ACC`	`200`	加速度，单位 100步/s²（最大 254）：200 → 约 0.14s 加到最快
`GRIP_TIME_MS`	`350`	夹爪开合等待时间
`ARM_SAFE_Z_MM`	`235.0f`	
`ARM_APPROACH_DZ_MM`	`35.0f`	取放前先停在目标上方这么高
`FOLD_YAW_RAW`	`1024`	
`FOLD_SHOULDER_RAW`	`3000`	
`FOLD_ELBOW_RAW`	`900`	
4. 物料与高度		
`MAT_HEIGHT_MM`	`60.0f`	初赛物料总高
`MAT_GRIP_FROM_BASE_MM`	`56.0f`	夹持点：顶部 Ø30 直段中心，离物料底面 56mm
`CAR_TRAY_FLOOR_Z_MM`	`92.0f`	车上载物槽底面离地高度
`TURNTABLE_TOP_Z_MM`	`90.0f`	【现场】原料转盘顶面高度（规则 80~100）
`FLOOR_Z_MM`	`0.0f`	
`SLOT0_X_MM`	`-50.0f`	
`SLOT0_Y_MM`	`-50.0f`	
`SLOT1_X_MM`	`45.0f`	
`SLOT1_Y_MM`	`50.0f`	
`SLOT2_X_MM`	`-50.0f`	
`SLOT2_Y_MM`	`50.0f`	
5. 比赛时间管理		
`MATCH_TIME_LIMIT_MS`	`180000`	每轮运行 3 分钟
`MATCH_RETURN_RESERVE_MS`	`6000`	剩余时间少于"预估返程时间+这个余量"就放弃剩下任务、直接回启停区
`TURNTABLE_WAIT_MAX_MS`	`20000`	转盘等某个颜色最多等多久（规则：停滞 15s+转盘 8s）
`QR_TIMEOUT_MS`	`6000`	读二维码超时

20　已知风险与下一步

风险	说明	建议
时间偏紧	模拟 150~178 秒，最后 1~3 个码垛常被放弃	实测稳定后适当调高 NAV_VMAX_MMPS、ARM_SPEED_RAW_PER_S；转盘等料时提前把机械臂停在取料点上方
陀螺仪接线松动	振动下杜邦线可能松，掉线后只能靠轮子	用螺丝端子或焊接；标好 ROT_SCALE 作后备
相机程序未写	协议已定；读码、颜色、圆环偏差都依赖它	尽快选型（OpenMV / K210 / MaixCam 等），按协议实现
启停区只有 2mm 余量	出发包络约 296×233mm，启停区 300×300	机械上把轮外侧宽度减到 280mm 以内
载物槽太大	内腔约 Ø82，物料底 Ø50，可滑动 ±16mm；夹爪对 Ø30 顶部只有约 ±8mm 容差	槽底加 Ø52 定位沉台
机械臂关节角度	够近处载物槽需要肩约 133°、肘约 -132°	实物确认不干涉；不行就调整槽位或臂长
障碍物	路线规划已支持"某段不通 → 绕行"，但还没有检测	相机或测距传感器检测，调用 field_block_edge()
规则边界	"粗加工区取回"与"不得再次移动已放置物料"的边界；圆环评分的偏差换算	向赛事方确认

建议的下一步顺序

1. 按第 17 章第 0~4 步，把底盘调通、标定好（只需要已到货的电机、驱动板、电池）。
2. 舵机到齐后做第 5 步，同时在实物上确认机械臂关节角度范围。
3. 选定相机型号，写相机端程序；选定屏幕，写显示后端。
4. 标定陀螺仪（第 17 章第 4.5 步）。
5. 在模拟场地上跑整场，记录每阶段用时和失败原因，针对性优化。

本讲解随工程 SmartCar_F407 一起提供。代码改动后，讲解中的代码片段可能与实际文件略有不同，以源码为准。